

□> **Templates** : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_2ème_année_FR_01.pdf`

Enseignant : BELACEL Madani

Niveau : ENS ^{2^{ème} ann^{ée}}

Dur^{ée} : 1h30 (90 min)

Public cible : Futurs enseignants de coll^{ège}/lyc^{ée} ^à D^épartement de fran^{çais}

Pr^érequis : Notions de base en Python, algorithmique, SQL et HTML/CSS

Modalit^é : TD en bin^{ômes} ^à alternance th^éorie et pratique sur machine

Objectif du TD

Pratiquer POO, algorithmes, SQL, Web, Git et conception d'une s^éance p^édagogique.

Objectifs op^érationnels : ^À l'issue de ce TD, l'^étudiant sera capable de :

1. Mod^{él}iser un syst^{ème} de notes avec les classes Python (instanciation, m^éthodes, attributs)
2. Impl^émenter l'algorithme de recherche dichotomique et justifier sa complexit^é en $O(\log n)$
3. ^Écrire une requ^{ête} SQL correcte avec jointure et agr^{ég}ation (SELECT, JOIN, GROUP BY, AVG)
4. Produire une page Web int^égrant un formulaire, une mise en page CSS et une interaction JavaScript
5. Appliquer un workflow Git de base (branche, commit, merge) et concevoir une s^éance p^édagogique

Niveau taxonomique de Bloom : Application (3) ^à Analyse (4) ^à Cr^éation (6)

Exercice 1 ^à POO Python (18 min)

Mise en situation :

Dans la vie d'un enseignant, le calcul des moyennes est une t^âche r^écurrente et fastidieuse. Un outil automatis^é permet non seulement de gagner du temps, mais aussi d'^éviter les erreurs de calcul. La programmation orient^{ée} objet (POO) est particuli^èrement adapt^{ée} pour mod^{él}iser ce probl^{ème} : une `Note` est un objet poss^{éd}ant des propri^{étés} (mati^{ère}, valeur, coefficient) et un `Bulletin` regroupe un ensemble de notes et sait en calculer la moyenne pond^{ér}ée.

Objectif sp^{éc}ifique : Comprendre et appliquer les concepts de classe, d'attribut, de m^éthode et de constructeur en Python.

^Ànonc^{er} : Cr^éer une classe `Note` (mati^{ère}, valeur, coefficient) et une classe `Bulletin` calculant la moyenne pond^{ér}ée.

Corrig^é d^{ét}ail^é :

```
class Note:
    def __init__(self, matiere, valeur, coef=1):
        self.matiere = matiere
        self.valeur = valeur
        self.coef = coef

class Bulletin:
    def __init__(self, notes):
        self.notes = notes
```

```
def moyenne(self):
    s = sum(n.valeur * n.coef for n in self.notes)
    c = sum(n.coef for n in self.notes)
    return s / c
```

Sous-questions :

1. Que se passe-t-il si la liste `notes` est vide dans `moyenne()` ? Proposer une gestion d'erreur.
2. Ajouter une méthode `afficher_bulletin()` qui imprime chaque note avec sa matière, sa valeur et son coefficient.
3. Comment garantir qu'une note est toujours comprise entre 0 et 20 ? Où placer cette validation ?
4. Implémenter une méthode `maximum()` qui retourne la note la plus élevée du bulletin.
5. Quelle est la différence entre un attribut d'instance et un attribut de classe ? Donner un exemple.

Variantes :

- Ajouter un attribut `appreciation` (chaîne) à `Note` et l'afficher dans le bulletin.
- Ajouter une classe `Eleve` qui possède un nom, un prénom et un bulletin. La moyenne s'obtient par `eleve.moyenne()`.
- Remplacer la liste de notes par un dictionnaire `{matiere: Note}` et gérer les matières obligatoires/optionnelles.

Différenciation pédagogique :

- Niveau 1 (soutien) : fournir un squelette avec les signatures des méthodes d'ajout ; l'étudiant remplit le corps.
- Niveau 2 (autonomie) : laisser l'étudiant concevoir librement les classes à partir de l'énoncé seul.
- Niveau 3 (approfondissement) : demander d'ajouter une méthode statique `Note.from_input()` qui construit une note à partir d'une saisie clavier.

Commentaires pas à pas du corrigé :

- `coef=1` rend le coefficient optionnel à valeur par défaut 1 si non fourni.
- Le constructeur de `Bulletin` reçoit une liste d'objets `Note` : relation de composition.
- `moyenne()` parcourt la liste avec une compréhension de générateur. `sum(...)` calcule la somme des produits `valeur * coef` (numérateur) et la somme des coefficients (dénominateur).
- Piège : oublier de gérer le cas `c == 0` (liste vide) à proposer `return 0.0` ou `raise ValueError("Aucune note")`.

Exercice 2 – Recherche dichotomique (18 min)

Mise en situation :

Imaginez que vous cherchez le mot « maison » dans un dictionnaire papier de 1000 pages : vous ouvrez au milieu, comparez, et décidez de chercher à gauche ou à droite. C'est le principe de la recherche dichotomique. Transposé en informatique, cet algorithme trouve un élément dans un tableau trié en temps logarithmique, ce qui le rend extrêmement efficace pour de grands volumes de données.

Objectif spécifique : Maîtriser la stratégie « diviser pour régner » et analyser la complexité algorithmique.

Nonc : Pseudo-code recherche dichotomique. Prcondition ? Complexité ?

Corrigé détaillé :

- Tableau **trié** obligatoire (prcondition essentielle).
- Diviser pour régner : comparer l'élément recherché avec la valeur au milieu du tableau, réduire l'intervalle de recherche à la moitié gauche ou droite.
- Complexité : $O(\log n)$ dans le pire des cas car chaque étape divise la taille du problème par 2.

Sous-questions :

1. Tracer manuellement la recherche de 12 dans [3, 7, 12, 18, 25, 30, 42]. Combien d'étapes nécessaires ?
2. Que se passe-t-il si le tableau n'est pas trié ? Donner un contre-exemple.
3. Écrire l'algorithme en Python (version itérative et version récursive).
4. Comparer le nombre d'opérations pour $n = 100$, 10 000 et 1 000 000 entre recherche linéaire et dichotomique.
5. Proposer une version qui retourne l'indice de la première occurrence en cas de doublons.

Variantes :

- Implémenter la version récursive de la recherche dichotomique.
- Adapter l'algorithme pour chercher dans un tableau d'objets (par exemple, chercher un élève par son nom dans une liste triée alphabétiquement).
- Implémenter la recherche par interpolation qui utilise la valeur plutôt que le milieu fixe pour deviner la position.

Différenciation pédagogique :

- Niveau 1 : fournir le pseudo-code avec des trous à compléter.
- Niveau 2 : demander l'implémentation complète en Python avec gestion des cas limites.
- Niveau 3 : ajouter une analyse comparative expérimentale (chronométrer les deux versions sur un grand tableau).

Commentaires pas à pas du corrigé :

- Étape 1 : vérifier que le tableau est trié (prcondition).
- Étape 2 : `debut = 0, fin = len(tableau) - 1`.
- Étape 3 : `tant que debut <= fin, calculer milieu = (debut + fin) // 2`.
- Étape 4 : `si tableau[milieu] == cible alors retourner milieu`.
- Étape 5 : `si tableau[milieu] < cible alors chercher à droite (debut = milieu + 1) ; sinon à gauche (fin = milieu - 1)`.
- Étape 6 : si la boucle se termine sans trouver alors retourner -1 ou None.

Exercice 3 – SQL (18 min)

Mise en situation :

Dans un Établissement scolaire, les notes des Élèves sont stockées dans une base de données relationnelle. Un enseignant peut avoir besoin d'extraire la moyenne de chaque Élève pour remplir un bulletin. La requête SQL appropriée combine les tables `eleve` et `note` via une jointure, puis agrège les résultats par Élève. Cet exercice simule une situation réelle d'interrogation de base de données.

Objectif spécifique : Maîtriser la syntaxe SQL des jointures et des fonctions d'agrégation avec GROUP BY.

À retenir : Tables `eleve(id, nom)` et `note(id_eleve, matiere, valeur)`. Requête : moyenne par Élève.

Corrigé détaillé :

```
SELECT e.nom, AVG(n.valeur) AS moyenne
FROM eleve e
JOIN note n ON e.id = n.id_eleve
GROUP BY e.id, e.nom;
```

Sous-questions :

1. Que renvoie la requête si un Élève n'a aucune note ? Comment modifier avec `LEFT JOIN` pour qu'il apparaisse avec une moyenne NULL ?
2. Ajouter une condition pour n'afficher que les Élèves ayant une moyenne supérieure à 10 (utiliser `HAVING`).
3. Trier le résultat par moyenne décroissante avec `ORDER BY`.
4. Écrire une requête qui donne la moyenne par matière pour toute la classe.
5. Proposer une sous-requête pour trouver l'Élève ayant la meilleure moyenne.

Variantes :

- Ajouter une colonne `prenom` à `eleve` et adapter la requête.
- Créer une table `matiere(id, nom, coefficient)` et pondérer les notes par matière.
- Écrire une requête qui calcule la moyenne globale, minimale et maximale sur toute la classe en une seule requête.

Différenciation pédagogique :

- Niveau 1 : fournir le schéma de la base et un jeu de données d'exemple ; Écrire la requête à taper par tape.
- Niveau 2 : faire varier les conditions (`WHERE`, `HAVING`, `ORDER BY`) et observer les changements.
- Niveau 3 : optimiser la requête avec des index et analyser le plan d'exécution (`EXPLAIN`).

Commentaires pas à pas du corrigé :

- `SELECT e.nom, AVG(n.valeur) AS moyenne` → alias `AS moyenne` pour la lisibilité.
- `FROM eleve e` → alias `e` pour simplifier l'écriture.
- `JOIN note n ON e.id = n.id_eleve` → jointure sur la clé étrangère.
- `GROUP BY e.id, e.nom` → toutes les colonnes non agrégées du `SELECT` doivent figurer dans le `GROUP BY`.
- Piège : oublier `e.id` dans le `GROUP BY` peut causer une erreur si deux Élèves ont le même nom.
- Alternative : `LEFT JOIN` pour inclure les Élèves sans note.

Exercice 4 – Page Web (18 min)

Mise en situation :

La création de formulaires Web est une compétence de base pour tout enseignant de technologie au collège. Un formulaire permet de collecter des données utilisateur (inscription, questionnaire, sondage). L'intégration harmonieuse du HTML (structure), du CSS (présentation) et du JavaScript (interactivité) constitue le socle du développement front-end.

Objectif spécifique : Créer une page Web interactive en intégrant les trois langages du Web front-end.

À retenir : Page HTML formulaire (nom, email) + CSS centré + JS alerte à la soumission.

Corrigé détaillé :

- `<form>` avec `input type="text"` et email.
- CSS `margin: auto; max-width: 400px;`
- `onsubmit` à `preventDefault()` + `alert('Merci')`.

Sous-questions :

1. Ajouter une validation JavaScript qui vérifie que le champ `nom` n'est pas vide avant soumission.
2. Ajouter un champ `message` (textarea) et un compteur de caractères en temps réel.
3. Rendre le formulaire responsive (adapté aux mobiles) avec une media query CSS.
4. Modifier l'alerte pour afficher le nom saisi : « Merci [nom] ! ».
5. Utiliser `localStorage` pour sauvegarder les données du formulaire et les réafficher après rechargement.

Variante :

- ... Ajouter un champ `âge` de type `number` avec validation (entre 6 et 99).
- ... Utiliser Bootstrap ou Tailwind CSS pour la mise en page au lieu du CSS pur.
- ... Envoyer les données du formulaire vers une API factice avec `fetch()` et afficher la réponse.

Différenciation pédagogique :

- Niveau 1 : fournir le squelette HTML complet ; l'étudiant ajoute uniquement le JavaScript.
- Niveau 2 : l'étudiant écrit le HTML, le CSS et le JS à partir d'une maquette (image).
- Niveau 3 : ajouter une validation asynchrone (vérifier si un nom d'utilisateur existe déjà).

Commentaires pas à pas du corrigé :

- HTML : `<form onsubmit="return handleSubmit(event)">` ou `onsubmit="return true"` pour éviter d'événement en JS.
- CSS : `display: flex; flex-direction: column; align-items: center; max-width: 400px; margin: auto;`
- JavaScript : `event.preventDefault()` empêche le rechargement ; `alert('Merci')` affiche une boîte de dialogue.
- Structure recommandée : séparer les fichiers (`index.html`, `style.css`, `script.js`) pour une meilleure organisation.

Déroulement à 90 min

Ex.1 18' | Ex.2 18' | Ex.3 18' | Ex.4 18' | Ex.5 13' | Clôture 5'

Rythme suggéré : 5 min de lecture/consigne, 10 min de travail autonome, 3 min de correction collective par exercice.

Diagrammes conceptuels

flowchart TD

A[Début du TD] --> B[Exercice 1
POO Python]

B --> C[Exercice 2
Recherche dichotomique]

C --> D[Exercice 3
SQL]

D --> E[Exercice 4
Page Web HTML/CSS/JS]

E --> F[Exercice 5
Git & Pédagogie]

F --> G[Clôture]

style A fill:#4a90d9,color:#fff

style B fill:#2ecc71,color:#fff

style C fill:#e67e22,color:#fff

style D fill:#e74c3c,color:#fff

style E fill:#9b59b6,color:#fff

style F fill:#2ecc71,color:#fff

style G fill:#4a90d9,color:#fff

flowchart LR

subgraph Compétences

direction TB

C1[POO
Classes & Objets] --> C2[Algorithmique
Dichotomie]

C2 --> C3[SQL
Jointure & Agrégation]

C3 --> C4[Web
HTML/CSS/JS]

C4 --> C5[Git & Pédagogie
Collaboration & Conception]

end

style C1 fill:#2ecc71,color:#fff

style C2 fill:#e67e22,color:#fff

style C3 fill:#e74c3c,color:#fff

style C4 fill:#9b59b6,color:#fff

style C5 fill:#4a90d9,color:#fff

mindmap

racine((TD Informatique))

POO

Classe Note

Classe Bulletin

Moyenne pondérée

Validation des notes

Algorithmes

Recherche dichotomique

Complexité $O(\log n)$

Tableau trié

Version récursive

SQL

Requête SELECT

Jointure JOIN

GROUP BY

HAVING, ORDER BY

Web

HTML formulaire

CSS centré

JavaScript alerte

Validation front-end

Git

Branches

Commits atomiques
Merge request
Résolution de conflits

Grille d'évaluation

Critère	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
POO : classe et moyenne	20 %	Classes absentes ou incorrectes	Classe partielle sans calcul	Classes correctes
Recherche dichotomique	20 %	Algorithme non maîtrisé	Logique partielle	Implémentation correcte
SQL : jointure et agrégation	20 %	Requête incorrecte	Syntaxe correcte mais résultat erroné	Requête correcte
Page Web : HTML/CSS/JS	20 %	Aucun rendu fonctionnel	Formulaire sans JS	Formulaire fonctionnel
Git et conception pédagogique	20 %	Workflow inconnu	Workflow partiel	Workflow complet

Barème de notation :

- 0-2/5 : Insuffisant → reprise obligatoire des concepts
- 2-3/5 : Partiel → consolidation nécessaire
- 3-4/5 : Satisfaisant → acquis utilisables en stage
- 4-5/5 : Excellent → prêt pour la transposition didactique

Note pédagogique ENS

Transposition didactique : Ce TD couvre cinq piliers fondamentaux de l'informatique (POO, algorithmique, SQL, Web, Git) qu'un futur enseignant de collège-lycée doit maîtriser pour transposer ces concepts dans ses cours de technologie. La diversité des exercices permet de montrer aux élèves comment les mêmes principes (abstraction, structuration, automatisation) s'appliquent à différents domaines.

Conseil pour l'enseignant formateur : Encouragez les étudiants à expliciter le lien entre chaque exercice et les programmes du secondaire. Par exemple, la recherche dichotomique peut être reformulée comme un jeu de "devine le nombre" au collège. Demandez-leur de produire une trace écrite réflexive après chaque exercice.

Liens avec les programmes du secondaire :

- 3e : algorithmes et programmation (Scratch et Python)
- 2nde : algorithmique, tableurs, HTML/CSS
- 1re Spéc NSI : POO, SQL, récursivité

Prolongement : Proposer une séance interdisciplinaire où les élèves de 3e créent un mini-projet utilisant formulaire Web, base de données et visualisation des notes avec un graphique.

Auto-évaluation pour l'étudiant ENS :

Après le TD, prendre 2 minutes pour cocher :

- [] Je sais créer une classe Python avec attributs et méthodes
- [] Je sais expliquer et implémenter la recherche dichotomique
- [] Je sais écrire une requête SQL avec jointure et GROUP BY

- [] Je sais créer un formulaire Web avec HTML/CSS/JS
- [] Je sais créer un workflow Git et concevoir une séance

Templates : Template/PG-fr.docx | Template/Presentation_BELACEL_V3.pptx

PDF : TD_Informatique_ENS_2ème_année_FR_02.pdf

TD Informatique â€” ENS 2^{ème} année â€” S^éance 02 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 2^{ème} année â€” D^épartement de fran^çais

Dur^ée : 1h30

Ann^ée universitaire : 2025-2026

Exercice 1 â€” D^éfinir une classe (15 min)

Ã€nonc^é : Cr^éer une classe `Livre` avec :

- Attributs : `titre`, `auteur`, `annee`, `nb_pages`
- M^éthode `__init__` avec tous les param^ètres
- M^éthode `afficher()` qui affiche les informations du livre
- M^éthode `est_long()` qui retourne `True` si le livre a plus de 300 pages

Corrig^é :

```
class Livre:
    def __init__(self, titre, auteur, annee, nb_pages):
        self.titre = titre
        self.auteur = auteur
        self.annee = annee
        self.nb_pages = nb_pages

    def afficher(self):
        print(f"'{self.titre}' â€” {self.auteur} ({self.annee}), {self.nb_pages}p.")

    def est_long(self):
        return self.nb_pages > 300

# Test
l = Livre("Informatique ENS", "Belacel", 2026, 250)
l.afficher()
print("Long ?", l.est_long()) # False
```

Exercice 2 â€” Attributs et m^éthodes (15 min)

Ã€nonc^é : Compl^éter la classe `CompteBancaire` :

```
class CompteBancaire:
    def __init__(self, titulaire, solde_initial=0):
        # Ã€ compléter : stocker le titulaire et le solde
        pass

    def deposer(self, montant):
        # Ã€ compléter : ajouter le montant au solde (vérifier que montant > 0)
        pass

    def retirer(self, montant):
        # Ã€ compléter : retirer si le solde est suffisant
        pass

    def afficher(self):
        # Ã€ compléter : afficher le titulaire et le solde
```

```
pass
```

Corrigé :

```
class CompteBancaire:
    def __init__(self, titulaire, solde_initial=0):
        self.titulaire = titulaire
        self.solde = solde_initial if solde_initial >= 0 else 0

    def deposer(self, montant):
        if montant > 0:
            self.solde += montant
            print(f"Dépôt de {montant} â€¸ effectu .")
        else:
            print("Montant invalide.")

    def retirer(self, montant):
        if 0 < montant <= self.solde:
            self.solde -= montant
            print(f"Retrait de {montant} â€¸ effectu .")
        else:
            print("Solde insuffisant ou montant invalide.")

    def afficher(self):
        print(f"Titulaire : {self.titulaire} â€¸ Solde : {self.solde} â€¸")
```

Exercice 3 â€¸ Variables de classe (15 min)

  non   : Cr  er une classe Etudiant avec :

- Variable de classe universite = "ENS Mostaganem"
- Variable de classe total_etudiants = 0
- Attributs d'instance : nom, groupe
- Le constructeur incr  mente total_etudiants
- M  thode __str__ qui retourne une cha  ne descriptive

Cr  er 3   tudiants et afficher leurs informations et le total.

Corrig   :

```
class Etudiant:
    universite = "ENS Mostaganem"
    total_etudiants = 0

    def __init__(self, nom, groupe):
        self.nom = nom
        self.groupe = groupe
        Etudiant.total_etudiants += 1

    def __str__(self):
        return f"{self.nom} â€¸ {self.groupe} ({self.universite})"

# Test
e1 = Etudiant("Ali", "ENS2-A")
e2 = Etudiant("Sara", "ENS2-A")
e3 = Etudiant("Mohamed", "ENS2-B")

print(e1) # Ali â€¸ ENS2-A (ENS Mostaganem)
print(e2)
print(e3)
```

```
print(f"Total : {Etudiant.total_etudiants} Ã©tudiants") # 3
```

Exercice 4 – Composition de classes (20 min)

À nonc : Créer deux classes :

1. Matière avec attributs nom, coefficient
 2. Évaluation avec attributs matière (objet Matière), note, date
- Méthode `ponderer()` qui retourne note – coefficient

Créer une matière « Informatique » (coef 3) et une Évaluation note 15. Afficher le résultat pondéré.

Corrigé :

```
class Matiere:
    def __init__(self, nom, coefficient):
        self.nom = nom
        self.coefficient = coefficient

    def __str__(self):
        return f"{self.nom} (coef {self.coefficient})"

class Evaluation:
    def __init__(self, matiere, note, date):
        self.matiere = matiere
        self.note = note
        self.date = date

    def ponderer(self):
        return self.note * self.matiere.coefficient

    def __str__(self):
        return f"{self.matiere.nom} : {self.note}/20 (coefficient {self.matiere.coefficient}) â†’ {self.ponderer()} pts"

# Test
info = Matiere("Informatique", 3)
evall = Evaluation(info, 15, "2026-03-15")
print(evall) # Informatique : 15/20 â†’ 45 pts
```

Exercice 5 – Classe avec liste d'objets (20 min)

À nonc : Créer une classe `Eleve` qui contient une liste d'Évaluations. Ajouter les méthodes :

- `ajouter_evaluation(matiere, note, date)` – créer une Évaluation et l'ajouter à la liste
- `moyenne()` – calculer la moyenne pondérée
- `afficher_releve()` – afficher toutes les Évaluations et la moyenne

Corrigé :

```
class Eleve:
    def __init__(self, nom, prenom):
        self.nom = nom
        self.prenom = prenom
        self.evaluations = []

    def ajouter_evaluation(self, matiere, note, date):
```

```

ev = Evaluation(matiere, note, date)
self.evaluations.append(ev)

def moyenne(self):
    if not self.evaluations:
        return 0.0
    total_pondere = sum(ev.ponderer() for ev in self.evaluations)
    total_coef = sum(ev.matiere.coefficient for ev in self.evaluations)
    return total_pondere / total_coef if total_coef > 0 else 0.0

def afficher_releve(self):
    print(f"=== Relevé de {self.prenom} {self.nom} ===")
    for ev in self.evaluations:
        print(f" {ev}")
    print(f" Moyenne générale : {self.moyenne():.2f}/20")

# Test
français = Matiere("Français", 2)
anglais = Matiere("Anglais", 1)

eleve = Eleve("Benali", "Sara")
eleve.ajouter_evaluation(info, 15, "2026-03-15")
eleve.ajouter_evaluation(français, 12, "2026-03-16")
eleve.ajouter_evaluation(anglais, 18, "2026-03-17")
eleve.afficher_releve()

```

Déroulement séance à 1h30 (90 min)

Phase	Durée	Total cumulé
Accueil	5 min	5 min
Exercice 1 à Classe Livre	15 min	20 min
Exercice 2 à CompteBancaire	15 min	35 min
Exercice 3 à Variables de classe	15 min	50 min
Exercice 4 à Composition	20 min	70 min
Exercice 5 à Liste d'objets	20 min	90 min
Total	90 min	

Diagrammes conceptuels

```

flowchart LR
    A[Classe] --> B[Attributs<br/>d'instance]
    A --> C[Attributs<br/>de classe]
    A --> D[Méthodes]
    B --> E[Livre<br/>CompteBancaire]
    C --> F[universite<br/>total_etudiants]
    D --> G[Composition]
    G --> H[Matiere + Evaluation<br/>à Eleve]

```

```

style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#2ecc71,color:#fff
style D fill:#e67e22,color:#fff
style G fill:#e74c3c,color:#fff
style H fill:#9b59b6,color:#fff

```

```

classDiagram
class Livre {
- titre: string
- auteur: string
- annee: int
- nb_pages: int
+ afficher()
+ est_long() bool
}
class CompteBancaire {
- titulaire: string
- solde: float
+ deposer(montant)
+ retirer(montant)
+ afficher()
}
class Etudiant {
+ universite: string
+ total_etudiants: int
- nom: string
- groupe: string
+ __str__() string
}
class Matiere {
- nom: string
- coefficient: int
}
class Evaluation {
- matiere: Matiere
- note: float
- date: string
+ ponderer() float
}
class Eleve {
- nom: string
- prenom: string
- evaluations: list
+ ajouter_evaluation()
+ moyenne() float
+ afficher_releve()
}
Matiere --> Evaluation : compos   dans
Evaluation --> Eleve : agr  g   dans

style Livre fill:#4a90d9,color:#fff
style CompteBancaire fill:#2ecc71,color:#fff
style Etudiant fill:#e67e22,color:#fff
style Matiere fill:#e74c3c,color:#fff
style Evaluation fill:#9b59b6,color:#fff
style Eleve fill:#4a90d9,color:#fff

```

Grille d'  valuation

Crit��re	Poids	Non acquis (0)	Partiel (0.5)
Cr��ation de classe (Livre)	20 %	Classe absente	Attributs partiellement d��finis
Attributs et m��thodes (CompteBancaire)	20 %	M��thodes non impl��ment��es	Logique partielle (d��p��t sans v��rification)
Variables de classe (Etudiant)	20 %	Variables non d��clar��es	Compteur non fonctionnel
Composition (Matiere + Evaluation)	20 %	Relation non mod��lis��e	Composition partielle sans m��thode pointer

Liste d'objets (Elevé)	20 %	Liste non utilisée	Moyenne non pondérée
------------------------	------	--------------------	----------------------

Note pédagogique ENS

Transposition didactique : Les concepts de classe et d'objet sont transposables au collège à travers la notion de « fiche modèle » : une classe est un modèle (template) dont chaque objet est une instance concrète (ex. : modèle de fiche «Ils ont des fiches remplies»). La composition peut être illustrée par un sac de cours contenant plusieurs matières.

Conseil pour l'enseignant formateur : Faites réaliser une affiche papier représentant une classe avec ses attributs et ses méthodes, puis demandez aux étudiants de « jouer » l'exécution d'une méthode en groupe. Cette approche kinesthésique renforce la compréhension des concepts abstraits.

Prolongement : Proposer un mini-projet de création d'un système de gestion de bibliothèque avec des classes Livre, Adherent et Emprunt, incluant la persistance des données dans un fichier JSON.

Templates : Template/PG-fr.docx | Template/Presentation_BELACEL_V3.pptx

PDF : TD_Informatique_ENS_2ème_année_FR_03.pdf

TD Informatique â€” ENS 2Ã¨me annÃ©e â€” SÃ©ance 03 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 2Ã¨me annÃ©e â€” DÃ©partement de franÃ§ais

DurÃ©e : 1h30

AnnÃ©e universitaire : 2025-2026

Exercice 1 â€” HÃ©ritage simple (15 min)

Ã‰noncÃ© : CrÃ©er une classe mÃ©re `Personne` et une classe fille `Enseignant` :

```
class Personne:
    def __init__(self, nom, prenom, email):
        # Ã‰complÃ©ter
        pass

    def se_presenter(self):
        return f"Je m'appelle {self.prenom} {self.nom}"

class Enseignant(Personne):
    def __init__(self, nom, prenom, email, matiere):
        # Ã‰complÃ©ter : appeler le constructeur parent ET stocker la matiÃ¨re
        pass

    def se_presenter(self):
        # Ã‰complÃ©ter : surcharger pour ajouter la matiÃ¨re
        pass
```

CorrigÃ© :

```
class Personne:
    def __init__(self, nom, prenom, email):
        self.nom = nom
        self.prenom = prenom
        self.email = email

    def se_presenter(self):
        return f"Je m'appelle {self.prenom} {self.nom}"

class Enseignant(Personne):
    def __init__(self, nom, prenom, email, matiere):
        super().__init__(nom, prenom, email)
        self.matiere = matiere

    def se_presenter(self):
        base = super().se_presenter()
        return f"{base} et j'enseigne la {self.matiere}"

# Test
prof = Enseignant("Belacel", "Madani", "m.belacel@ens.dz", "Informatique")
print(prof.se_presenter())
# Je m'appelle Madani Belacel et j'enseigne l'Informatique
```

Exercice 2 â€” Surcharge et `super()` (15 min)

Ã‰noncÃ© : CrÃ©er une hiÃ©rarchie de trois niveaux :

1. `Animal` : attribut `nom`, méthode `parler()` â†’ retourne `"..."`
2. `Chien(Animal)` : surcharge `parler()` â†’ retourne `"Woof !"`
3. `ChienDeGarde(Chien)` : surcharge `parler()` â†’ utilise `super().parler()` puis ajoute `"MÃ©chant chien !"`

CrÃ©er un `ChienDeGarde` nommé `"Rex"` et appeler `parler()`.

CorrigÃ© :

```
class Animal:
    def __init__(self, nom):
        self.nom = nom

    def parler(self):
        return "..."

class Chien(Animal):
    def parler(self):
        return "Woof !"

class ChienDeGarde(Chien):
    def parler(self):
        return super().parler() + " MÃ©chant chien !"

# Test
rex = ChienDeGarde("Rex")
print(f"{rex.nom} dit : {rex.parler()}")
# Rex dit : Woof ! MÃ©chant chien !
```

Exercice 3 â€” Polymorphisme (20 min)

Ã‰noncÃ© : CrÃ©er une fonction `faire_parler(animal)` qui appelle la mÃ©thode `parler()` de n'importe quel animal. Puis crÃ©er :

- `Chat(Animal)` avec `parler()` â†’ `"Miaou !"`
- `Vache(Animal)` avec `parler()` â†’ `"Meuh !"`
- `Canard(Animal)` avec `parler()` â†’ `"Coin coin !"`

Tester la fonction avec les trois animaux.

CorrigÃ© :

```
class Chat(Animal):
    def parler(self):
        return "Miaou !"

class Vache(Animal):
    def parler(self):
        return "Meuh !"

class Canard(Animal):
    def parler(self):
        return "Coin coin !"

def faire_parler(animal):
    """Fonction polymorphe : Accepte n'importe quel Animal"""
    print(f"{animal.nom} dit : {animal.parler()}")

# Test
animaux = [
    Chat("Minou"),
```

```
Vache("Marguerite"),
Canard("Donald"),
ChienDeGarde("Rex")
]
```

```
for a in animaux:
    faire_parler(a)
```

R  sultat :

```
Minou dit : Miaou !
Marguerite dit : Meuh !
Donald dit : Coin coin !
Rex dit : Woof ! M  chant chien !
```

Exercice 4 – Encapsulation (20 min)

  nonc   : Cr  er une classe `CompteEpargne` avec :

- Attribut prot  g   `_taux` (taux d'int  r  t annuel en %, ex: 2.5)
- Attribut priv   `__solde` (ne peut   tre modifi   directement)
- Getter `solde` avec `@property`
- Setter `solde` avec `@solde.setter` (v  rifie que le montant      0)
- M  thode `appliquer_interets()` qui ajoute les int  r  ts au solde

Corrig   :

```
class CompteEpargne:
    def __init__(self, titulaire, solde=0, taux=2.5):
        self.titulaire = titulaire
        self._taux = taux
        self.__solde = 0
        if solde >= 0:
            self.__solde = solde

    @property
    def solde(self):
        return self.__solde

    @solde.setter
    def solde(self, montant):
        if montant >= 0:
            self.__solde = montant
        else:
            print("Erreur : le solde ne peut pas   tre n  gatif.")

    def appliquer_interets(self):
        interets = self.__solde * (self._taux / 100)
        self.__solde += interets
        print(f"Int  r  ts de {interets:.2f}      appliqu  s (taux {self._taux}%).")

    def __str__(self):
        return f"{self.titulaire}      {self.__solde:.2f}     "

# Test
ce = CompteEpargne("Ali", 1000, 2.5)
print(ce) # Ali      1000.00     
ce.appliquer_interets() # Int  r  ts de 25.00      appliqu  s
print(ce) # Ali      1025.00     
# ce.__solde = 9999 # Erreur ! Attribut priv   inaccessible
```

```
print(ce.solde) # 1025.0 (via getter)
```

Exercice 5 – Hiérarchie scolaire complète (20 min)

À noter : Créer les classes suivantes avec héritage :

1. MembreENS : nom, prenom, email
2. Etudiant (MembreENS) : groupe, liste de notes
 - ajouter_note(note), moyenne(), __str__()
3. Professeur (MembreENS) : matiere, liste de cours
 - ajouter_cours(cours), __str__()
4. Tester avec un Etudiant et un professeur

Corrigé :

```
class MembreENS:
    def __init__(self, nom, prenom, email):
        self.nom = nom
        self.prenom = prenom
        self.email = email

    def __str__(self):
        return f"{self.prenom} {self.nom}"

class Etudiant(MembreENS):
    def __init__(self, nom, prenom, email, groupe):
        super().__init__(nom, prenom, email)
        self.groupe = groupe
        self.notes = []

    def ajouter_note(self, note):
        if 0 <= note <= 20:
            self.notes.append(note)

    def moyenne(self):
        return sum(self.notes) / len(self.notes) if self.notes else 0.0

    def __str__(self):
        return f"{super().__str__()} â€” {self.groupe} (Moy: {self.moyenne():.2f})"

class Professeur(MembreENS):
    def __init__(self, nom, prenom, email, matiere):
        super().__init__(nom, prenom, email)
        self.matiere = matiere
        self.cours = []

    def ajouter_cours(self, cours):
        self.cours.append(cours)

    def __str__(self):
        return f"{super().__str__()} â€” {self.matiere} ({len(self.cours)} cours)"

# Test
etu = Etudiant("Benali", "Sara", "s.benali@ens.dz", "ENS2-A")
etu.ajouter_note(15)
etu.ajouter_note(12)
etu.ajouter_note(18)
print(etu) # Sara Benali â€” ENS2-A (Moy: 15.00)
```

```

prof = Professeur("Belacel", "Madani", "m.belacel@ens.dz", "Informatique")
prof.ajouter_cours("POO")
prof.ajouter_cours("BD")
print(prof) # Madani Belacel â€” Informatique (2 cours)

```

DÃ©roulement sÃ©ance â€” 1h30 (90 min)

Phase	DurÃ©e	Total cumulÃ©
Accueil	5 min	5 min
Exercice 1 â€” HÃ©ritage simple	15 min	20 min
Exercice 2 â€” Surcharge et super()	15 min	35 min
Exercice 3 â€” Polymorphisme	20 min	55 min
Exercice 4 â€” Encapsulation	20 min	75 min
Exercice 5 â€” HiÃ©rarchie complÃ©te	15 min	90 min
Total	>90 min	

Diagrammes conceptuels

```

flowchart TD
    A[HÃ©ritage] --> B[Classe mÃ©re<br/>Personne / Animal]
    A --> C[Surcharge<br/>super()]
    A --> D[Polymorphisme]
    A --> E[Encapsulation]
    D --> F[faire_parler()<br/>une interface]
    E --> G[Attributs privÃ©s<br/>__solde]
    E --> H[Getters/Setters<br/>@property]
    B --> I[HiÃ©rarchie complÃ©te<br/>MembreENS â†’ Etudiant / Professeur]

```

```

style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#2ecc71,color:#fff
style D fill:#e67e22,color:#fff
style E fill:#e74c3c,color:#fff
style I fill:#9b59b6,color:#fff

```

```

classDiagram
class Animal {
- nom: string
+ parler() string
}
class Chien {
+ parler() string
}
class ChienDeGarde {
+ parler() string
}
class Chat {
+ parler() string
}
class Vache {
+ parler() string
}
class Canard {
+ parler() string
}

```

```

}
Animal <|-- Chien
Animal <|-- Chat
Animal <|-- Vache
Animal <|-- Canard
Chien <|-- ChienDeGarde

class MembreENS {
- nom: string
- prenom: string
- email: string
}
class Etudiant {
- groupe: string
- notes: list
+ ajouter_note(note)
+ moyenne() float
}
class Professeur {
- matiere: string
- cours: list
+ ajouter_cours(cours)
}
MembreENS <|-- Etudiant
MembreENS <|-- Professeur

style Animal fill:#4a90d9,color:#fff
style Chien fill:#2ecc71,color:#fff
style ChienDeGarde fill:#e67e22,color:#fff
style Chat fill:#e74c3c,color:#fff
style Vache fill:#9b59b6,color:#fff
style Canard fill:#4a90d9,color:#fff
style MembreENS fill:#2ecc71,color:#fff
style Etudiant fill:#e67e22,color:#fff
style Professeur fill:#e74c3c,color:#fff

```

Grille d'Évaluation

Critère	Poids	Non acquis (0)	Partiel (0.5)
Héritage simple (Personne à l'Enseignant)	20 %	Héritage non compris	Appel super() manquant
Surcharge et super() (Animal à l'ChienDeGarde)	20 %	Chaine d'héritage absente	super() mal utilisé
Polymorphisme (faire_parler)	20 %	Notion non maîtrisée	Fonction sans typage polymorphique
Encapsulation (CompteEpargne)	20 %	Attributs tous publics	Property sans setter
Hiérarchie complète (MembreENS)	20 %	Aucune hiérarchie	Hiérarchie partielle

Note pédagogique ENS

Transposition didactique : L'héritage peut être introduit au collège par la notion de « catégories et sous-catégories » (ex. : Animal à l'Mammifère à l'Chien), ce qui correspond à la pensée classificatoire enseignée en sciences. Le polymorphisme s'illustre par une même consigne (« fais-toi présenter ») qui produit des résultats différents selon l'objet.

Conseil pour l'enseignant formateur : Utilisez des supports visuels (cartes de classe, diagrammes de hiérarchie au tableau) avant de passer au code. Proposez un exercice d'branchement où les

Étudiants incarnent des objets et exécutent des méthodes en suivant le chaînage d'héritage.

Prolongement : Étudier le patron de conception « Template Method » ou « Strategy » en montrant comment l'héritage et le polymorphisme permettent de concevoir des frameworks réutilisables.

Templates : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_2ème_année_FR_04.pdf`

TD Informatique â€” ENS 2^{ème} année â€” S^éance 04 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 2^{ème} année â€” D^épartement de fran^çais

Dur^ée : 1h30

Ann^ée universitaire : 2025-2026

Public cible : Futurs enseignants de coll^{ège}/lyc^{ée}

Pr^érequis : Notions de base en Python, boucles, fonctions

Notions abord^{ées} : R^écursivit^é, factorielle, Fibonacci, dichotomie, tri bulle, tri fusion

Exercice 1 â€” R^écursivit^é : factorielle (15 min)

Mise en situation : La factorielle ($n! = 1 \times 2 \times \dots \times n$) appara^{ît} en combinatoire. Sa d^éfinition r^écursive ($n! = n \times (n-1)!$) est l'exemple canonique pour introduire la r^écursivit^é.

Objectif sp^{éc}ifique : Comprendre le m^écanisme d'appel r^écursif â€” cas de base et appel r^écursif â€” et savoir tracer la pile d'ex^écution.

Ã‰nonc^é : Ã‰crire une fonction r^écursive `factorielle(n)` et tracer son ex^écution pour $n=4$.

Corrig^é :

```
def factorielle(n):
    if n <= 1: # Cas de base
        return 1
    return n * factorielle(n - 1) # Appel récursif
```

Trac^é pour $n = 4$:

```
factorielle(4)
â†’ 4 * factorielle(3)
â†’ 4 * (3 * factorielle(2))
â†’ 4 * (3 * (2 * factorielle(1)))
â†’ 4 * (3 * (2 * 1)) â†’ 4 * (3 * 2) â†’ 4 * 6 â†’ 24
```

Table de tra^çage (pile d'appels) :

Niveau	Appel	n	Retour	Action
0	<code>factorielle(4)</code>	4	?	Lancement
1	<code>factorielle(3)</code>	3	?	Appel (n-1)
2	<code>factorielle(2)</code>	2	?	Appel (n-1)
3	<code>factorielle(1)</code>	1	1	Cas de base
2	<code>factorielle(2)</code>	2	$2! = 2$	Retour
1	<code>factorielle(3)</code>	3	$3! = 6$	Retour
0	<code>factorielle(4)</code>	4	$4! = 24$	R ^é sultat

Sous-questions : 1. Tracer pour $n=5$? 2. `factorielle(-1)` â€” protection ? 3. Version it^érative ? 4. Complexit^é temporelle/spatiale ? 5. Arbre d'appels pour $n=4$?

Variante : `def somme_recursive(n):` `def lru_cache` `def` Sans multiplication explicite.

Différenciation : N1 : squelette fourni. N2 : tracer puis implémenter. N3 : `sys.setrecursionlimit` et stack overflow.

Complexité : Temps $O(n)$, Espace $O(n)$. Version itérative : $O(n)$ temps, $O(1)$ espace.

Exercice 2 « Récursivité : Fibonacci (20 min)

Mise en situation : La suite de Fibonacci illustre le danger de la récursivité naïve : le double appel récursif cause une explosion exponentielle.

Objectif spécifique : Analyser la complexité exponentielle et découvrir les optimisations (itératif, mémorisation).

Nonc : Écrire une fonction récursive `fibonacci(n)` (`F(0)=0`, `F(1)=1`, `F(n)=F(n-1)+F(n-2)`). Calculer `fibonacci(7)` à la main.

Corrigé :

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
```

Valeurs :

`fib(0)=0 fib(1)=1 fib(2)=1 fib(3)=2 fib(4)=3 fib(5)=5 fib(6)=8 fib(7)=13`

Arbre d'appels pour fibonacci(5) « 15 appels :

```
fib(5)
/ \
fib(4) fib(3)
/ \ / \
fib(3) fib(2) fib(2) fib(1)
/ \ / \
fib(2) fib(1) ... (15 appels)
```

Soit 15 appels pour `fib(5)`, 41 pour `fib(7)`, 331M pour `fib(40)` !

Comparaison : Naïve $O(2^n)$, itérative $O(n)$, mémorisée $O(n)$. Pour $n=30$: 2,6M vs 29 vs 30 opérations.

Sous-questions : 1. Nombre d'appels pour `fib(10)` ? 2. Version itérative ? 3. Version avec dictionnaire `memo`. 4. Pour $n=100$, version naïve impossible ? 5. `@lru_cache`.

Variante : itératif simple `def` `@functools.lru_cache` `def` Exponentiation de matrice $O(\log n)$.

Différenciation : N1 : tracer pour $n=4$. N2 : analyser complexité et comparer. N3 : chronométrer les 3 versions.

Exercice 3 « Recherche dichotomique (20 min)

Mise en situation : Avec des données triées, on peut éliminer la moitié des données à chaque étape : complexité $O(\log n)$.

Objectif spécifique : Implémenter la recherche dichotomique et analyser sa complexité logarithmique.

Exercice 1 : Fonction dichotomie : Appliquer la fonction `dichotomie` sur la liste `notes = [5, 8, 10, 12, 15, 18, 20]` pour trouver 15.

Corrigé :

```
def dichotomie(tableau, cible):
    debut, fin = 0, len(tableau) - 1
    while debut <= fin:
        milieu = (debut + fin) // 2
        if tableau[milieu] == cible:
            return milieu
        elif tableau[milieu] < cible:
            debut = milieu + 1
        else:
            fin = milieu - 1
    return -1

notes = [5, 8, 10, 12, 15, 18, 20]
print(dichotomie(notes, 15)) # 4
```

Table de traçage pour cible = 15 (trouvée) :

Itér.	debut	fin	milieu	tab[milieu]	Comparaison	Action
1	0	6	3	12	12 < 15	debut = 4
2	4	6	5	18	18 > 15	fin = 4
3	4	4	4	15	15 == 15	Trouvée !

Table de traçage pour cible = 7 (non trouvée) :

Itér.	debut	fin	milieu	tab[milieu]	Comparaison	Action
1	0	6	3	12	12 > 7	fin = 2
2	0	2	1	8	8 > 7	fin = 0
3	0	0	0	5	5 < 7	debut = 1
4	1	0	-1	-1	debut > fin	Retour -1

Sous-questions : 1. Tracer pour 18. Combien d'itérations ? 2. Que se passe-t-il en cas de doublons ? 3. Version récursive ? 4. Pour $n=1024$, nombre max d'itérations ? 5. Version générique (str, float...).

Variante : Retourner l'indice d'insertion. Recherche par interpolation. Recherche avec fonction `key`.

Différenciation : N1 : pseudo-code uniquement. N2 : implémentation Python complète. N3 : comparer expérimentalement avec la recherche linéaire.

Gabarit à compléter :

```
def dichotomie(tableau, cible):
    debut = 0
    fin = len(tableau) - 1
    while debut ... fin:
        milieu = (debut + fin) // ...
        if tableau[milieu] == cible:
            return ...
        elif tableau[milieu] ... cible:
            debut = milieu + 1
        else:
            fin = ... - 1
```

```
return ...
```

Complexité : $O(\log n)$. Chaque étape divise la taille par 2.

Exercice 4 – Tri bulle (20 min)

Mise en situation : Le tri bulle compare les éléments adjacents et les échange si nécessaire. Les plus grands « remontent » vers la fin comme des bulles.

Objectif spécifique : Comprendre le tri par échanges successifs et analyser sa complexité $O(n^2)$.

Nonc : Fonction `tri_bulle(tableau)`. Tracer sur [15, 8, 12, 20, 5].

Corrigé :

```
def tri_bulle(tableau):
    n = len(tableau)
    for i in range(n):
        trie = True
        for j in range(n - i - 1):
            if tableau[j] > tableau[j + 1]:
                tableau[j], tableau[j + 1] = tableau[j + 1], tableau[j]
                trie = False
        if trie:
            break
    return tableau
```

Tracé :

```
Départ : [15, 8, 12, 20, 5]
Pass 1 : [8, 12, 15, 5, 20] Pass 2 : [8, 12, 5, 15, 20]
Pass 3 : [8, 5, 12, 15, 20] Pass 4 : [5, 8, 12, 15, 20]
```

Table de traçage :

Pass	Comparaisons	Échanges	Tableau
1	15>8, 15>12, 15>20?, 20>5	3	[8, 12, 15, 5, 20]
2	8>12?, 12>15?, 15>5	1	[8, 12, 5, 15, 20]
3	8>12?, 12>5	1	[8, 5, 12, 15, 20]
4	8>5	1	[5, 8, 12, 15, 20]
5	trie=True → break	0	Trié

Sous-questions : 1. Combien de comparaisons dans le pire cas ? 2. Rôle du drapeau `trie` ? 3. Tracer sur [1, 2, 3, 4, 5] (d'abord trié). 4. Adapter en ordre décroissant. 5. Version sans `trie` → différence ?

Variante : Sans optimisation → Tri cocktail → Sur liste chaînée.

Différenciation : N1 : code fourni, tracer manuellement. N2 : implémenter avec optimisation. N3 : compter les comparaisons expérimentalement.

Gabarit à compléter :

```
def tri_bulle(tableau):
    n = len(tableau)
    for i in range(n):
```

```

trie = True
for j in range(...):
    if tableau[j] > tableau[j + 1]:
        tableau[j], tableau[j + 1] = tableau[j + 1], tableau[j]
    trie = ...
    if trie:
        ...
return tableau

```

Complexité : Pire $O(n^2)$, Meilleur $O(n)$ (dans le meilleur des cas). En moyenne $O(n \log n)$.

Exercice 5 – Tri fusion (15 min)

Mise en situation : Le tri fusion illustre la « diviser pour régner » : découper récursivement en deux, puis fusionner les parties triées. Complexité $O(n \log n)$.

Objectif spécifique : Maîtriser la conception d'un algorithme récursif et analyser sa complexité.

À retenir : Écrire `tri_fusion(tableau)` et `fusionner(gauche, droite)`. Trier [15, 8, 12, 20, 5, 10, 18].

Corrigé :

```

def tri_fusion(tableau):
    if len(tableau) <= 1:
        return tableau
    milieu = len(tableau) // 2
    gauche = tri_fusion(tableau[:milieu])
    droite = tri_fusion(tableau[milieu:])
    return fusionner(gauche, droite)

def fusionner(gauche, droite):
    resultat = []
    i = j = 0
    while i < len(gauche) and j < len(droite):
        if gauche[i] <= droite[j]:
            resultat.append(gauche[i]); i += 1
        else:
            resultat.append(droite[j]); j += 1
    resultat.extend(gauche[i:]); resultat.extend(droite[j:])
    return resultat

notes = [15, 8, 12, 20, 5, 10, 18]
print(tri_fusion(notes))

```

Tracé du découpage :

```

[15, 8, 12, 20, 5, 10, 18]
  [15, 8, 12, 20]
    [15, 8] et [12, 20]
      [15] et [8]
      [12] et [20]
      [5, 10, 18]
        [5]
        [10, 18] et [18]
        [5, 10, 18]
        [5, 8, 12, 15, 18, 20]

```

Table des fusions :

Type	Gauche	Droite	Résultat
------	--------	--------	----------

1	[15]	[8]	[8, 15]
2	[12]	[20]	[12, 20]
3	[8, 15]	[12, 20]	[8, 12, 15, 20]
4	[5]	[10]	[5, 10]
5	[5, 10]	[18]	[5, 10, 18]
6	[8, 12, 15, 20]	[5, 10, 18]	[5, 8, 10, 12, 15, 18, 20]

Sous-questions : 1. Nombre de niveaux de récursion pour n ? 2. Pourquoi le tri fusion est-il stable ? 3. Modifier pour ordre décroissant. 4. Comparer comparaisons tri fusion vs tri bulle pour $n=10, 100, 1000$. 5. Mémoire utilisée par le tri fusion ?

Variantes : Tracer sur 4 éléments la liste chaînée sur liste chaînée. Version itérative bottom-up.

Différenciation : N1 : fournir `fusionner()`, demander `tri_fusion()`. N2 : les deux fonctions. N3 : compteur de comparaisons.

Gabarit à compléter :

```
def tri_fusion(tableau):
    if len(tableau) <= 1: return ...
    milieu = len(tableau) // 2
    gauche = tri_fusion(tableau[:milieu])
    droite = tri_fusion(tableau[milieu:])
    return fusionner(gauche, droite)

def fusionner(gauche, droite):
    resultat = []
    i = j = 0
    while i < len(gauche) and j < len(droite):
        if gauche[i] <= droite[j]:
            resultat.append(gauche[i]); i += 1
        else:
            resultat.append(droite[j]); j += 1
    resultat.extend(gauche[i:]); resultat.extend(droite[j:])
    return resultat
```

Complexité : $O(n \log n)$ dans tous les cas. Espace $O(n)$. Plus efficace que le tri bulle.

Déroulement séance 1h30 (90 min)

Phase	Durée	Total cumulé
Accueil	5 min	5 min
Exercice 1 : Factorielle	15 min	20 min
Exercice 2 : Fibonacci	20 min	40 min
Exercice 3 : Dichotomie	20 min	60 min
Exercice 4 : Tri bulle	20 min	80 min
Exercice 5 : Tri fusion	10 min	90 min
Total	>90 min	

Diagrammes conceptuels

```

flowchart TD
A[Algorithmes de recherche et tri] --> B[RÃ©cursivitÃ©]
A --> C[Recherche]
A --> D[Tri]
B --> E[Factorielle<br/>rÃ©cursive]
B --> F[Fibonacci<br/>rÃ©cursif]
C --> G[Dichotomie<br/> $O(\log n)$ ]
D --> H[Tri bulle<br/> $O(n^2)$ ]
D --> I[Tri fusion<br/> $O(n \log n)$ ]

```

```

style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#e67e22,color:#fff
style D fill:#e74c3c,color:#fff
style I fill:#9b59b6,color:#fff

```

```

flowchart LR
subgraph RÃ©cursivitÃ©
F1[factorielle(4)] --> F2[4 Ã— factorielle(3)]
F2 --> F3[3 Ã— factorielle(2)]
F3 --> F4[2 Ã— factorielle(1)]
F4 --> F5[1]
end

```

```

subgraph Comparaison
B1[Tri bulle  $O(n^2)$ ] --> B2["n=100 : 10 000 op"]
M1[Tri fusion  $O(n \log n)$ ] --> M2["n=100 : ~460 op"]
end

```

```

style F1 fill:#4a90d9,color:#fff
style F5 fill:#2ecc71,color:#fff
style B1 fill:#e74c3c,color:#fff
style M1 fill:#2ecc71,color:#fff

```

```

flowchart TD
subgraph Factorielle
A1[factorielle(4)] --> A2[factorielle(3)]
A2 --> A3[factorielle(2)]
A3 --> A4[factorielle(1)]
A4 --> A5[R=1] --> A6[R=2] --> A7[R=6] --> A8[R=24]
end
subgraph Dichotomie
D1[DÃ©but] --> D2{debut Ã© fin ?}
D2 -->|Oui| D3[milieu]
D3 --> D4{tab[milieu]=cible?}
D4 -->|Oui| D5[Retour]
D4 -->|Non| D6{< cible ?}
D6 -->|Oui| D7[debut=milieu+1]
D6 -->|Non| D8[fin=milieu-1]
D7 --> D2; D8 --> D2
D2 -->|Non| D9[-1]
end
subgraph Tri_fusion
T1[tab] --> T2{len Ã© 1?}
T2 -->|Oui| T3[Retour]
T2 -->|Non| T4[DÃ©couper]
T4 --> T5[tri_fusion gauche]
T4 --> T6[tri_fusion droite]
T5 --> T7[fusionner]; T6 --> T7
T7 --> T8[Retour]
end
style A8 fill:#2ecc71,color:#fff
style D5 fill:#2ecc71,color:#fff
style T8 fill:#2ecc71,color:#fff

```

```

flowchart TD
    subgraph Tri_bulle
        B1[Début] --> B2[Pour i]
        B2 --> B3[trie=True]
        B3 --> B4[Pour j]
        B4 --> B5{tab[j] > tab[j+1]?}
        B5 -->|Oui| B6[changer]
        B6 --> B7[trie=False]; B7 --> B4
        B5 -->|Non| B4
        B4 --> B8{trie == True?}
        B8 -->|Oui| B9[Break]; B8 -->|Non| B2
        B9 --> B10[Retour]
    end
    end
    style B10 fill:#2ecc71,color:#fff

```

Grille d'évaluation

Critère	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
Factorielle	20 %	Non récursif	Cas de base mal défini	Récursivité correcte avec tracé
Fibonacci	20 %	Double appel non compris	Résultats partiels	Implémentation et conscience complexité
Dichotomie	20 %	Non fonctionnel	Logique partielle	Correcte avec indices
Tri bulle	20 %	Non fonctionnel	Sans optimisation	Avec drapeau et tracé
Tri fusion	20 %	Fusion non implémente	Découpage seul	Complet avec complexité

Barème : 0-2/5 Insuffisant € 2-3/5 Partiel € 3-4/5 Satisfaisant € 4-5/5 Excellent

Note pédagogique ENS

Transposition didactique : La récursivité peut être introduite au collège par les poupées russes ou les fractales. Les algorithmes de tri peuvent être simulés avec des cartes numérotées déplacées sur la table (approche kinesthésique).

Conseil pour l'enseignant formateur : Après chaque algorithme, demandez aux étudiants de l'exécuter à la main sur un petit jeu de données avant de le coder. Insistez sur la comparaison des complexités.

Activités débranchées (CS Unplugged) :

- Factorielle : combien de façons de ranger 4 objets dans une file ?
- Dichotomie : jeu « Devine le nombre » entre 1 et 100
- Tri bulle : objets avec des pancartes numérotées se déplacent
- Tri fusion : diviser la classe en deux groupes, chaque groupe trie ses cartes

Prolongement : Défi d'optimisation € implémenter Fibonacci itératif (programmation dynamique) et comparer les temps avec la version récursive naïve pour $n=30, 35, 40$.

Templates : Template/PG-fr.docx | Template/Presentation_BELACEL_V3.pptx

PDF : TD_Informatique_ENS_2ème_année_FR_05.pdf

TD Informatique â€” ENS 2^{ème} année â€” S^éance 05 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 2^{ème} année â€” D^épartement de fran^çais

Dur^ée : 1h30

Ann^ée universitaire : 2025-2026

Exercice 1 â€” CREATE TABLE et INSERT (20 min)

Ã‰nonc^é : Soit le sch^éma relationnel d'une biblioth^éque :

- Auteur (id_auteur INT PRIMARY KEY, nom VARCHAR(50), prenom VARCHAR(50), annee_naissance INT)
- Livre (id_livre INT PRIMARY KEY, titre VARCHAR(100), annee_publication INT, id_auteur INT REFERENCES Auteur(id_auteur))

1. Ã‰crire les instructions CREATE TABLE

2. Ins^érer 3 auteurs et 4 livres

3. Afficher tous les livres

Corrig^é :

```
CREATE TABLE Auteur (
  id_auteur INT PRIMARY KEY,
  nom VARCHAR(50),
  prenom VARCHAR(50),
  annee_naissance INT
);
```

```
CREATE TABLE Livre (
  id_livre INT PRIMARY KEY,
  titre VARCHAR(100),
  annee_publication INT,
  id_auteur INT REFERENCES Auteur(id_auteur)
);
```

```
INSERT INTO Auteur VALUES
(1, 'Dijkstra', 'Edsger', 1930),
(2, 'Knuth', 'Donald', 1938),
(3, 'Turing', 'Alan', 1912);
```

```
INSERT INTO Livre VALUES
(1, 'The Art of Computer Programming', 1968, 2),
(2, 'A Discipline of Programming', 1976, 1),
(3, 'Computing Machinery and Intelligence', 1950, 3),
(4, 'Selected Writings on Computing', 1982, 1);
```

```
SELECT * FROM Livre;
```

Exercice 2 â€” SELECT et WHERE (15 min)

Ã‰nonc^é : Sur la table Livre ci-dessus, Ã©crire les requ^êtes suivantes :

1. Titres des livres publi^és apr^{ès} 1970

2. Livres de l'auteur id_auteur = 1 (Dijkstra)

3. Livres dont le titre contient "Programming"

Corrigé :

```
-- 1. Livres après 1970
SELECT titre, annee_publication
FROM Livre
WHERE annee_publication > 1970;

-- 2. Livres de Dijkstra
SELECT titre
FROM Livre
WHERE id_auteur = 1;

-- 3. Titre contient "Programming"
SELECT titre, annee_publication
FROM Livre
WHERE titre LIKE '%Programming%';
```

Exercice 3 – Jointure avec INNER JOIN (20 min)

À noncer : Enrichir la base avec une table Emprunt (id_emprunt INT PRIMARY KEY, id_livre INT REFERENCES Livre, date_emprunt DATE, nom_emprunteur VARCHAR(50)).

Insérer 5 emprunts puis écrire les requêtes :

1. Afficher chaque emprunt avec le titre du livre et le nom de l'auteur
2. Compter le nombre d'emprunts par auteur
3. Trouver le(s) livre(s) jamais emprunté(s)

Corrigé :

```
CREATE TABLE Emprunt (
  id_emprunt INT PRIMARY KEY,
  id_livre INT REFERENCES Livre(id_livre),
  date_emprunt DATE,
  nom_emprunteur VARCHAR(50)
);

INSERT INTO Emprunt VALUES
(1, 1, '2026-01-15', 'Ali'),
(2, 1, '2026-02-10', 'Sara'),
(3, 3, '2026-03-05', 'Mohamed'),
(4, 2, '2026-03-12', 'Ali'),
(5, 4, '2026-04-01', 'Sara');

-- 1. Emprunts avec titre et auteur
SELECT e.nom_emprunteur, l.titre, a.nom || ' ' || a.prenom AS auteur, e.date_emprunt
FROM Emprunt e
INNER JOIN Livre l ON e.id_livre = l.id_livre
INNER JOIN Auteur a ON l.id_auteur = a.id_auteur;

-- 2. Nombre d'emprunts par auteur
SELECT a.nom || ' ' || a.prenom AS auteur, COUNT(e.id_emprunt) AS nb_emprunts
FROM Auteur a
LEFT JOIN Livre l ON a.id_auteur = l.id_auteur
LEFT JOIN Emprunt e ON l.id_livre = e.id_livre
GROUP BY a.id_auteur, auteur;
```

```
-- 3. Livres jamais empruntés
SELECT l.titre
FROM Livre l
LEFT JOIN Emprunt e ON l.id_livre = e.id_livre
WHERE e.id_emprunt IS NULL;
```

Exercice 4 – Mise à jour et suppression (15 min)

Nonc : Écrire les requêtes SQL pour :

1. Ajouter une colonne disponible BOOLEAN à Livre
2. Mettre à jour la colonne : TRUE si le livre n'a aucun emprunt en cours
3. Supprimer un auteur et ses livres (CASCADE)

Corrigé :

```
-- 1. Ajouter colonne
ALTER TABLE Livre ADD COLUMN disponible BOOLEAN DEFAULT TRUE;

-- 2. Mettre à jour (livres ayant des emprunts = non disponibles)
UPDATE Livre SET disponible = FALSE
WHERE id_livre IN (SELECT DISTINCT id_livre FROM Emprunt);

-- Vérification
SELECT titre, disponible FROM Livre;

-- 3. Supprimer avec CASCADE (si contrainte déclarée ON DELETE CASCADE)
-- ALTER TABLE Livre ADD CONSTRAINT fk_auteur
-- FOREIGN KEY (id_auteur) REFERENCES Auteur(id_auteur) ON DELETE CASCADE;
-- DELETE FROM Auteur WHERE id_auteur = 3; -- supprime aussi ses livres
```

Exercice 5 – Modélisation complète (20 min)

Nonc : Concevoir le schéma relationnel pour une gestion d'étudiants et de leurs inscriptions aux cours avec notes :

1. Créer les tables Etudiant, Cours, Inscription
2. Insérer 3 étudiants, 2 cours, 5 inscriptions
3. Afficher pour chaque étudiant : son nom, les cours suivis et la note

Corrigé :

```
CREATE TABLE Etudiant (
  id_etudiant INT PRIMARY KEY,
  nom VARCHAR(50),
  prenom VARCHAR(50),
  email VARCHAR(100)
);

CREATE TABLE Cours (
  id_cours INT PRIMARY KEY,
  intitule VARCHAR(100),
  credit INT
);

CREATE TABLE Inscription (
  id_etudiant INT REFERENCES Etudiant(id_etudiant),
  id_cours INT REFERENCES Cours(id_cours),
```

```

note DECIMAL(4,2),
PRIMARY KEY (id_etudiant, id_cours)
);

INSERT INTO Etudiant VALUES
(1, 'Benali', 'Sara', 's.benali@ens.dz'),
(2, 'Ziani', 'Ahmed', 'a.ziani@ens.dz'),
(3, 'Kaci', 'Leila', 'l.kaci@ens.dz');

INSERT INTO Cours VALUES
(101, 'POO', 4),
(102, 'BD', 3);

INSERT INTO Inscription VALUES
(1, 101, 15.00),
(1, 102, 12.50),
(2, 101, 17.00),
(2, 102, 14.00),
(3, 101, 10.50);

-- Relevé complet
SELECT e.nom || ' ' || e.prenom AS etudiant,
c.intitule AS cours,
i.note
FROM Inscription i
JOIN Etudiant e ON i.id_etudiant = e.id_etudiant
JOIN Cours c ON i.id_cours = c.id_cours
ORDER BY etudiant, cours;

-- Moyenne par Étudiant
SELECT e.nom || ' ' || e.prenom AS etudiant,
ROUND(AVG(i.note), 2) AS moyenne
FROM Inscription i
JOIN Etudiant e ON i.id_etudiant = e.id_etudiant
GROUP BY e.id_etudiant, etudiant;

```

Déroulement séance 1h30 (90 min)

Phase	Durée	Total cumulé
Accueil	5 min	5 min
Exercice 1 : CREATE / INSERT	20 min	25 min
Exercice 2 : SELECT / WHERE	15 min	40 min
Exercice 3 : Jointures	20 min	60 min
Exercice 4 : UPDATE / DELETE	15 min	75 min
Exercice 5 : Modélisation	15 min	90 min
Total	90 min	

Diagrammes conceptuels

```

flowchart TD
    A[Base de données relationnelle] --> B[Création<br/>CREATE TABLE]
    A --> C[Insertion<br/>INSERT]
    A --> D[Interrogation<br/>SELECT]
    A --> E[Mise à jour<br/>UPDATE / ALTER]

```

```

A --> F[Suppression<br/>DELETE / CASCADE]
D --> G[Jointures<br/>INNER / LEFT JOIN]
D --> H[Filtrage<br/>WHERE / LIKE]
D --> I[Aggrégation<br/>GROUP BY / COUNT]

```

```

style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#2ecc71,color:#fff
style D fill:#e67e22,color:#fff
style E fill:#e74c3c,color:#fff
style F fill:#e74c3c,color:#fff
style G fill:#9b59b6,color:#fff

```

```

erDiagram
Auteur ||--o{ Livre : "crit"
Livre ||--o{ Emprunt : "concerne"
Etudiant ||--o{ Inscription : "effectue"
Cours ||--o{ Inscription : "contient"

```

```

Auteur {
int id_auteur PK
string nom
string prenom
int annee_naissance
}
Livre {
int id_livre PK
string titre
int annee_publication
int id_auteur FK
}
Emprunt {
int id_emprunt PK
int id_livre FK
date date_emprunt
string nom_emprunteur
}
Etudiant {
int id_etudiant PK
string nom
string prenom
string email
}
Cours {
int id_cours PK
string intitule
int credit
}
Inscription {
int id_etudiant FK
int id_cours FK
decimal note
}

style Auteur fill:#4a90d9,color:#fff
style Livre fill:#2ecc71,color:#fff
style Emprunt fill:#e67e22,color:#fff
style Etudiant fill:#9b59b6,color:#fff
style Cours fill:#e74c3c,color:#fff
style Inscription fill:#4a90d9,color:#fff

```

Grille d'Évaluation

Critère	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
Création de tables et insertion	20 %	Tables non créées	INSERT incomplet	Création et insertion
SELECT et filtrage WHERE	20 %	Syntaxe SELECT inconnue	Filtrage partiel	Requêtes avec WHERE
Jointures (INNER / LEFT JOIN)	20 %	Jointure non comprise	INNER JOIN seul	Jointures multiples correctes
Mise à jour et suppression	20 %	ALTER / UPDATE inconnus	UPDATE sans sous-requête	UPDATE, ALTER et DELETE
Modélisation complète	20 %	Schéma non normalisé	Tables sans clés étrangères	Schéma complet avec normalisation

Note pédagogique ENS

Transposition didactique : La modélisation relationnelle peut être introduite au collège par la création de tableaux à double entrée (lignes = enregistrements, colonnes = attributs), ce que les élèves manipulent déjà en mathématiques. Les jointures s'illustrent par le « jeu des mots croisés » où l'on relie deux grilles par une information commune.

Conseil pour l'enseignant formateur : Commencez par un exercice « papier-crayon » de modélisation avant de passer au SQL. Demandez aux étudiants de dessiner le schéma entité-association au tableau et de vérifier les cardinalités. Insistez sur la différence entre WHERE (filtrage sur ligne) et HAVING (filtrage sur groupe).

Prolongement : Proposer un projet de création d'une petite base de données pour la bibliothèque de l'établissement avec une interface Web simple en PHP ou Python Flask permettant de lister, ajouter et rechercher des livres.

Templates : Template/PG-fr.docx | Template/Presentation_BELACEL_V3.pptx

PDF : TD_Informatique_ENS_2ème_année_FR_06.pdf

TD Informatique â€” ENS 2^{ème} année â€” S^éance 06 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 2^{ème} année â€” D^épartement de fran^çais

Dur^ée : 1h30

Ann^ée universitaire : 2025-2026

Exercice 1 â€” Structure HTML de base (15 min)

Ã‰nonc^é : Cr^éer une page HTML `presentation.html` qui pr^ésente votre CV en ligne. Inclure :

- Titre "Mon CV"
- En-t^{ête} avec votre nom (balise `<h1>`)
- Une photo de profil fictive (image placeholder)
- Une section "Ã‰ propos" en paragraphe
- Une liste de comp^étences (non ordonn^ée)
- Un pied de page avec votre email

Corrig^é :

```
<!DOCTYPE html>
<html lang="fr">
<head>
<meta charset="UTF-8">
<title>Mon CV</title>
</head>
<body>
<header>
<h1>Sara Benali</h1>

</header>

<section>
<h2>Ã‰ propos</h2>
<p>Ã‰tudiante en 2ème année Ã l'ENS Mostaganem. Passionnée d'informatique et de pédagogique.</p>
</section>

<section>
<h2>Compétences</h2>
<ul>
<li>Python, Java, SQL</li>
<li>Développement Web (HTML, CSS, JavaScript)</li>
<li>Systèmes d'exploitation (Linux, Windows)</li>
</ul>
</section>

<footer>
<p>Contact : s.benali@ens.dz</p>
</footer>
</body>
</html>
```

Exercice 2 – CSS : mise en page (20 min)

À faire : Ajouter un fichier `style.css` interne à la page précédente pour :

- Centrer le contenu max-width 800px
- Changer la couleur de fond du header (bleu clair)
- Styliser les sections avec bordure et marges
- Utiliser une police Google Fonts (Roboto)

Corrigé :

```
<!DOCTYPE html>
<html lang="fr">
<head>
<meta charset="UTF-8">
<title>Mon CV</title>
<link href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap"
rel="stylesheet">
<style>
* {
margin: 0;
padding: 0;
box-sizing: border-box;
}

body {
font-family: 'Roboto', sans-serif;
max-width: 800px;
margin: 0 auto;
padding: 20px;
background-color: #f5f5f5;
}

header {
background-color: #4a90d9;
color: white;
padding: 30px;
text-align: center;
border-radius: 10px;
margin-bottom: 20px;
}

header img {
border-radius: 50%;
margin-top: 10px;
}

section {
background-color: white;
border: 1px solid #ddd;
border-radius: 8px;
padding: 20px;
margin-bottom: 20px;
box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

h2 {
color: #4a90d9;
margin-bottom: 10px;
}

ul {
list-style-type: square;
```

```
padding-left: 20px;
}

footer {
text-align: center;
color: #888;
font-size: 0.9em;
}
</style>
</head>
<body>
<header>
<h1>Sara Benali</h1>

</header>

<section>
<h2>À propos</h2>
<p>Étudiante en 2ème année à l'ENS Mostaganem.</p>
</section>

<section>
<h2>Compétences</h2>
<ul>
<li>Python, Java, SQL</li>
<li>Développement Web (HTML, CSS, JavaScript)</li>
<li>Systèmes d'exploitation</li>
</ul>
</section>

<footer>
<p>Contact : s.benali@ens.dz</p>
</footer>
</body>
</html>
```

Exercice 3 – JavaScript : DOM et Événements (20 min)

À noter : Ajouter un script JavaScript à la page CV pour :

1. Un bouton "Afficher/Masquer" les compétences (toggle)
2. Un compteur de visites stocké dans localStorage
3. Affichage de la date et heure actuelle dans le footer

Corrigé :

```
<script>
// 1. Toggle compétences
function toggleCompetences() {
const section = document.querySelector('section:nth-of-type(2)');
if (section.style.display === 'none') {
section.style.display = 'block';
btn.textContent = 'Masquer les compétences';
} else {
section.style.display = 'none';
btn.textContent = 'Afficher les compétences';
}
}

const btn = document.createElement('button');
btn.textContent = 'Masquer les compétences';
```

```

btn.onclick = toggleCompetences;
document.body.insertBefore(btn, document.querySelector('footer'));

// 2. Compteur de visites
let visites = localStorage.getItem('cv_visites') || 0;
visites++;
localStorage.setItem('cv_visites', visites);
const compteur = document.createElement('p');
compteur.textContent = `Visites : ${visites}`;
document.querySelector('footer').appendChild(compteur);

// 3. Horloge
function afficherHeure() {
  const now = new Date();
  const horloge = document.getElementById('horloge');
  if (horloge) {
    horloge.textContent = now.toLocaleString('fr-FR');
  }
}
// Ajouter dans footer
const horloge = document.createElement('p');
horloge.id = 'horloge';
document.querySelector('footer').appendChild(horloge);
setInterval(afficherHeure, 1000);
afficherHeure();
</script>

```

Exercice 4 – Formulaire avec validation (20 min)

À réaliser : Créer une page HTML `contact.html` avec un formulaire de contact :

- Champs : nom, email, message
- Validation JavaScript : nom non vide, email valide (contient @), message à 10 caractères
- Afficher les erreurs sous chaque champ
- Au succès, afficher "Merci !" dans une boîte

Corrigé :

```

<!DOCTYPE html>
<html lang="fr">
<head>
<meta charset="UTF-8">
<title>Contact</title>
<style>
body { font-family: Arial, sans-serif; max-width: 600px; margin: 50px auto; }
label { display: block; margin-top: 10px; font-weight: bold; }
input, textarea { width: 100%; padding: 8px; margin-top: 5px; }
.error { color: red; font-size: 0.9em; }
.success { background: #d4edda; padding: 20px; border-radius: 5px; }
button { margin-top: 15px; padding: 10px 20px; background: #4a90d9; color: white; border: none; cursor: pointer; }
</style>
</head>
<body>
<h1>Contactez-nous</h1>
<form id="contactForm">
<label>Nom :</label>
<input type="text" id="nom">
<div class="error" id="nomError"></div>

```

```

<label>Email :</label>
<input type="email" id="email">
<div class="error" id="emailError"></div>

<label>Message :</label>
<textarea id="message" rows="5"></textarea>
<div class="error" id="messageError"></div>

<button type="submit">Envoyer</button>
</form>
<div id="success" style="display:none;" class="success"><h2>Merci !</h2></div>

<script>
document.getElementById('contactForm').onsubmit = function(e) {
e.preventDefault();

// R  initialiser
document.querySelectorAll('.error').forEach(el => el.textContent = '');
document.getElementById('success').style.display = 'none';

const nom = document.getElementById('nom').value.trim();
const email = document.getElementById('email').value.trim();
const message = document.getElementById('message').value.trim();
let valide = true;

if (!nom) {
document.getElementById('nomError').textContent = 'Le nom est requis.';
valide = false;
}

if (!email || !email.includes('@')) {
document.getElementById('emailError').textContent = 'Email invalide.';
valide = false;
}

if (message.length < 10) {
document.getElementById('messageError').textContent = 'Le message doit contenir au moins 10 caract  res.';
valide = false;
}

if (valide) {
document.getElementById('contactForm').reset();
document.getElementById('success').style.display = 'block';
}
};
</script>
</body>
</html>

```

Exercice 5 – Mini-projet : To-do list interactive (15 min)

  nonc   : Cr  er une to-do list avec HTML + CSS + JS dans un seul fichier. Fonctionnalit  s :

- Champ de saisie + bouton "Ajouter"
- Cliquer sur un   l  ment le marque comme termin   (barr  )
- Bouton supprimer    c  t   de chaque t  che

Corrig   :

```

<!DOCTYPE html>
<html lang="fr">
<head>
<meta charset="UTF-8">
<title>To-Do List</title>
<style>
body { font-family: Arial; max-width: 500px; margin: 50px auto; }
.ajout { display: flex; gap: 10px; margin-bottom: 20px; }
input { flex: 1; padding: 10px; }
button { padding: 10px 20px; cursor: pointer; }
ul { list-style: none; padding: 0; }
li { display: flex; justify-content: space-between; align-items: center;
padding: 10px; background: #f9f9f9; margin: 5px 0; border-radius: 5px; cursor: pointer; }
li.fait { text-decoration: line-through; color: #888; background: #e9e9e9; }
li.suppr { background: #dc3545; color: white; border: none; padding: 5px 10px; cursor:
pointer; border-radius: 3px; }
</style>
</head>
<body>
<h1>To-Do List</h1>
<div class="ajout">
<input type="text" id="tacheInput" placeholder="Nouvelle tâche...">
<button onclick="ajouterTache()">Ajouter</button>
</div>
<ul id="listeTaches"></ul>

<script>
function ajouterTache() {
const input = document.getElementById('tacheInput');
const texte = input.value.trim();
if (!texte) return;

const li = document.createElement('li');
li.textContent = texte;

li.onclick = function(e) {
if (e.target !== btn) this.classList.toggle('fait');
};

const btn = document.createElement('button');
btn.textContent = '✕';
btn.className = 'suppr';
btn.onclick = function(e) {
e.stopPropagation();
li.remove();
};

li.appendChild(btn);
document.getElementById('listeTaches').appendChild(li);
input.value = '';
input.focus();
}

document.getElementById('tacheInput').addEventListener('keypress', function(e) {
if (e.key === 'Enter') ajouterTache();
});
</script>
</body>
</html>

```

Grille d'Évaluation

Critère	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
---------	-------	----------------	---------------	------------

Structure HTML (titres, sections, footer)	20 %	Absence d'Ã©lÃ©ments requis	Partiellement complÃ©te	Structure complÃ©te
Mise en page CSS	20 %	Aucun style appliquÃ©	Style partiel	Design cohÃ©rent
InteractivitÃ© JavaScript (DOM)	20 %	Aucun script	Scripts partiels	Toggle, compteur,
Validation de formulaire	20 %	Aucune validation	Validation partielle	Validation complÃ©te
To-do list interactive	20 %	Non fonctionnelle	Partiellement fonctionnelle	Ajout, marquage, s

Note pÃ©dagogique ENS

Transposition didactique : Les concepts de HTML/CSS/JS peuvent Ãªtre transposÃ©s au collÃ©ge en introduisant d'abord la structure d'une page web avec HTML simple, puis la mise en forme avec CSS sans JavaScript. En lycÃ©e, on ajoute l'interactivitÃ© avec JavaScript pour crÃ©er des pages dynamiques et des formulaires.

Conseil pour l'enseignant formateur : Insister sur la sÃ©paration des prÃ©occupations (structure/style/comportement) et sur les bonnes pratiques d'accessibilitÃ©. Proposer des exercices progressifs : d'abord reproduire des maquettes, puis crÃ©er librement.

Prolongement : Demander aux Ã©tudiants d'intÃ©grer une API externe (ex. mÃ©tÃ©o) ou de publier leur CV en ligne via GitHub Pages.

Diagrammes conceptuels

Diagramme 1 â€” Structure de la page CV

```
graph TD
  A[Page CV] --> B[En-tÃªte]
  A --> C[Section Ã€ propos]
  A --> D[Section CompÃ©tences]
  A --> E[Pied de page]
  B --> F[Titre h1]
  B --> G[Photo de profil]
  D --> H[Python]
  D --> I[Java]
  D --> J[SQL]
  D --> K[Web]
  style A fill:#4a90d9,color:#fff
  style B fill:#2ecc71,color:#fff
  style C fill:#e67e22,color:#fff
  style D fill:#e74c3c,color:#fff
  style E fill:#9b59b6,color:#fff
```

Diagramme 2 â€” Technologies Web

```
mindmap
  root((DÃ©veloppement Web))
    HTML
      Structure
      Balises
      SÃ©mantique
      CSS
      Mise en page
      Couleurs
      Polices
      Responsive
      JavaScript
      DOM
      Ã©vÃ©nements
      localStorage
      Formulaires
```

Outils
 Navigateur
 Éditeur de code
 GitHub Pages

Déroulement séance de 1h30 (90 min)

Phase	Durée	Total cumulé
Accueil	5 min	5 min
Exercice 1 : Structure HTML	15 min	20 min
Exercice 2 : CSS	20 min	40 min
Exercice 3 : JavaScript DOM	20 min	60 min
Exercice 4 : Formulaire validation	20 min	80 min
Exercice 5 : To-do list	10 min	90 min
Total	90 min	

Templates : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_2ème_année_FR_07.pdf`

TD Informatique â€” ENS 2^{me} annÃ©e â€” SÃ©ance 07 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 2^{me} annÃ©e â€” DÃ©partement de franÃ§ais

DurÃ©e : 1h30

AnnÃ©e universitaire : 2025-2026

Exercice 1 â€” Initialisation et premier commit (15 min)

Ã‰noncÃ© : Simuler les commandes Git pour :

1. Initialiser un dÃ©pÃ´t Git
2. CrÃ©er un fichier README.md
3. Ajouter le fichier Ã l'index (staging)
4. Faire un commit avec le message "Initial commit"

CorrigÃ© :

```
# 1. Initialiser le dÃ©pÃ´t
git init

# 2. CrÃ©er le fichier
echo "# Projet ENS2" > README.md
echo "TD Git - Cycle PÃ©dagogique" >> README.md

# 3. Ajouter Ã l'index
git add README.md

# 4. Premier commit
git commit -m "Initial commit"

# VÃ©rifier
git log --oneline
```

Arbre aprÃ¨s Exercice 1 :

```
* (HEAD -> master) Initial commit
```

Exercice 2 â€” Travail avec les branches (20 min)

Ã‰noncÃ© : Simuler le workflow suivant :

1. CrÃ©er une branche developement
2. Basculer sur cette branche
3. CrÃ©er TP1.md et TP2.md (chacun avec une ligne de contenu)
4. Ajouter et commiter les deux fichiers (message : "Ajout TP1 et TP2")
5. Rebaser sur master â€” constater que les fichiers n'y sont pas

CorrigÃ© :

```
# 1-2. CrÃ©er et basculer
git checkout -b developement
```

```
# 3. Créer les fichiers
echo "# TP1 - POO" > TP1.md
echo "# TP2 - BD" > TP2.md

# 4. Commit
git add TP1.md TP2.md
git commit -m "Ajout TP1 et TP2"

# 5. Retour sur master
git checkout master

# Vérifier
ls # TP1.md et TP2.md ne sont PAS visibles
git log --oneline --all # Montre les deux commits
```

Arbre après Exercice 2 :

```
* (developpement) Ajout TP1 et TP2
* (HEAD -> master) Initial commit
```

Exercice 3 – Fusion (merge) et conflit (20 min)

À noter : Simuler un conflit de fusion :

1. Sur master, modifier README.md : ajouter la ligne "## Description"
2. Committer : `git add . && git commit -m "Modif README sur master"`
3. Sur developpement, modifier README.md : ajouter la ligne "## Auteur: Sara"
4. Committer : `git add . && git commit -m "Modif README sur dev"`
5. Fusionner developpement dans master à l'issue d'un CONFLIT
6. Résoudre le conflit manuellement

Corrigé :

```
# Sur master
echo -e "# Projet ENS2\n## Description" > README.md
git add README.md && git commit -m "Modif README sur master"

# Sur developpement
git checkout developpement
echo -e "# Projet ENS2\n## Auteur: Sara" > README.md
git add README.md && git commit -m "Modif README sur dev"

# Fusion
git checkout master
git merge developpement
# CONFLIT dans README.md

# Résoudre : Éditer README.md pour combiner les deux
# Contenu final :
: # Projet ENS2
: ## Description
: ## Auteur: Sara

git add README.md
git commit -m "Résolution conflit README"
```

Arbre après Exercice 3 :

```
* (HEAD -> master) R  solution conflit README
|\
| * Modif README sur dev
* | Modif README sur master
|/
* (developpement) Ajout TP1 et TP2
* Initial commit
```

Exercice 4 – Git flow : rebase et reset (20 min)

  nonc   : Simuler un git-flow simplifi   :

1. Cr  er une branche `feature/calcul-moyenne` depuis `developpement`
2. Ajouter `moyenne.py` (fonction simple), commiter
3. Avant de fusionner, master a avanc   (simuler un commit)
4. Rebaser `feature/calcul-moyenne` sur `master`

Corrig   :

```
# 1. Cr  er la branche feature
git checkout developpement
git checkout -b feature/calcul-moyenne

# 2. Cr  er et commiter la fonction
echo "def moyenne(notes): return sum(notes)/len(notes)" > moyenne.py
git add moyenne.py && git commit -m "Ajout fonction moyenne"

# 3. Simuler un commit sur master
git checkout master
echo "Fichier suppl  mentaire" > note.txt
git add note.txt && git commit -m "Ajout note.txt"

# 4. Rebase
git checkout feature/calcul-moyenne
git rebase master
# Les commits feature sont rejou  s par-dessus master

# Alternative : git merge master (fusion classique)
```

Arbre avant rebase :

```
* (master) Ajout note.txt
| * (HEAD -> feature/calcul-moyenne) Ajout fonction moyenne
|/
* (developpement) Ajout TP1 et TP2
| ...
```

Arbre apr  s rebase :

```
* (HEAD -> feature/calcul-moyenne) Ajout fonction moyenne
* (master) Ajout note.txt
| ...
```

Exercice 5 – Workflow collaboratif complet (15 min)

  nonc   :   crire la s  quence compl  te de commandes pour le sc  nario suivant :

Deux d  veloppeurs (Ali et Sara) travaillent sur le m  me d  p  t GitHub `ens-project`.

1. Ali cr  e le d  p  t, ajoute un README, commit, push sur `main`

2. Sara clone le dépôt distant
3. Ali crée une branche `feature/auth` et y travaille
4. Sara crée une branche `feature/ui` et y travaille
5. Ali finit le premier, merge dans `main` via Pull Request
6. Sara rebase sa branche sur `main`, puis Pull Request

Corrigé :

```
# 1. Ali : dépôt distant
# (Crée sur github.com/ali/ens-project)
git init
echo "# Projet ENS" > README.md
git add README.md && git commit -m "Initial"
git remote add origin https://github.com/ali/ens-project.git
git push -u origin main

# 2. Sara : clone
git clone https://github.com/ali/ens-project.git

# 3. Ali : feature/auth
git checkout -b feature/auth
echo "def login(): pass" > auth.py
git add auth.py && git commit -m "Ajout auth"
git push -u origin feature/auth
# à l'issue Pull Request sur GitHub

# 4. Sara : feature/ui
git checkout -b feature/ui
echo "<h1>Bienvenue</h1>" > index.html
git add index.html && git commit -m "Page accueil"
git push -u origin feature/ui

# 5. Ali : Pull Request merge sur main
git checkout main
git merge feature/auth
git push origin main

# 6. Sara : rebase sur main
git checkout main
git pull origin main
git checkout feature/ui
git rebase main
# Résoudre conflits éventuels
git push -f origin feature/ui
# à l'issue Pull Request
```

Grille d'évaluation

Critère	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
Initialisation et premier commit	20 %	Impossible d'initialiser ou commiter	Étapes réalisées avec aide	Init, ad
Gestion des branches	20 %	Ne crée pas de branche	Crée mais ne bascule pas correctement	Crée
Fusion et résolution de conflits	20 %	Ne sait pas fusionner	Fusion simple sans conflit	Résol
Rebase et reset	20 %	Concepts non compris	Rebase exécuté avec aide	Rebas
Workflow collaboratif complet	20 %	Workflow non maîtrisé	Partiellement compris	Clone,

Note pédagogique ENS

Transposition didactique : Les concepts de versionnement peuvent être introduits au collège via des exercices de suivi de modifications de texte (ex. rédaction collaborative). En lycée, on peut simuler Git avec des scénarios concrets de travail en équipe avant d'utiliser le terminal.

Conseil pour l'enseignant formateur : Utiliser des analogies (ex. "Git est comme un lecteur de sauvegarde de jeu vidéo") et privilégier la pratique sur des répertoires locaux avant d'introduire le distant. Le conflit de fusion est l'étape la plus délicate ; la préparer avec un exercice papier.

Prolongement : Proposer un projet collectif où chaque étudiant contribue sur une branche différente, avec revue de code via Pull Request sur GitHub.

Diagrammes conceptuels

Diagramme 1 – Workflow Git

```
graph LR
  A[Working Directory] -->|git add| B[Staging Area]
  B -->|git commit| C[Dépôt Local]
  C -->|git push| D[Dépôt Distant]
  D -->|git clone| E[Clone local]
  C -->|git branch| F[Branche]
  F -->|git merge| C
  F -->|git rebase| C
  style A fill:#4a90d9,color:#fff
  style B fill:#2ecc71,color:#fff
  style C fill:#e67e22,color:#fff
  style D fill:#e74c3c,color:#fff
  style E fill:#9b59b6,color:#fff
  style F fill:#4a90d9,color:#fff
```

Diagramme 2 – Stratégie de branches

```
gitGraph
  commit
  branch developement
  checkout developement
  commit id: "Ajout TP1 et TP2"
  checkout main
  commit id: "Modif README"
  checkout developement
  commit id: "Modif README dev"
  checkout main
  merge developement id: "Résolution conflit"
  branch feature/auth
  checkout feature/auth
  commit id: "Ajout auth.py"
  checkout main
  commit id: "Ajout note.txt"
  checkout feature/auth
  rebase main id: "Rebase sur main"
```

Déroulement séance – 1h30 (90 min)

Phase	Durée	Total cumulé
Accueil	5 min	5 min
Exercice 1 – Init / premier commit	15 min	20 min
Exercice 2 – Branches	20 min	40 min

Exercice 3 : Merge + conflit	20 min	60 min
Exercice 4 : Rebase et reset	20 min	80 min
Exercice 5 : Workflow complet	10 min	90 min
Total	90 min	

Templates : Template/PG-fr.docx | Template/Presentation_BELACEL_V3.pptx

PDF : TD_Informatique_ENS_2022_me_année_FR_08.pdf

TD Informatique â€” ENS 2^{ème} année â€” Séance 08 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 2^{ème} année â€” Département de français

Durée : 1h30

Année universitaire : 2025-2026

Exercice 1 â€” Analyse d'un exercice Scratch (20 min)

À noter : Un enseignant propose l'exercice Scratch suivant à des élèves de 1^{ère} année moyenne :

"Le chat Scratch doit avancer de 50 pas, dire 'Bonjour !' pendant 2 secondes, puis reculer de 50 pas."

1. Quels blocs Scratch sont nécessaires ?
2. Quelles sont les difficultés cognitives pour un élève débutant ?
3. Proposer une adaptation pour un élève en difficulté.

Corrigé :

1. Blocs nécessaires :

- avancer de 50 pas (bloc mouvement)
- dire "Bonjour !" pendant 2 secondes (bloc apparence)
- avancer de -50 pas **ou** reculer de 50 pas
- drapeau vert cliqué (déclencheur)

2. Difficultés cognitives :

- **Nombres négatifs :** les élèves peuvent ne pas comprendre que -50 pas = reculer
- **Séquençage :** respecter l'ordre des instructions (lecture séquentielle)
- **Concepts temporels :** "pendant 2 secondes" est un temps d'attente, pas une durée d'exécution
- **Abstraction :** l'écran représente un espace mathématique (coordonnées)

3. Adaptation (difficulté) :

- Version simplifiée sans nombres négatifs :

`

avancer de 50 pas

dire "Bonjour !" pendant 2 secondes

attendre 2 secondes

avancer de 50 pas (vers la droite)

`

- Utiliser des couleurs pour chaque type de bloc
- Étiqueter chaque bloc avec une consigne en français

Exercice 2 – Concevoir une fiche de séance (25 min)

À noter : Concevoir la **fiche de séance** pour une séance d'initiation à la programmation avec Scratch destinée à des élèves de 1^{ère} année moyenne (11-12 ans). Durée : 1h00.

La fiche doit inclure :

- Objectif général
- Objectifs spécifiques (3)
- Prérequis
- Matériel
- Déroulement (4 phases, avec durées)
- Évaluation
- Prolongement

Corrigé :

FICHE DE SÉANCE – Initiation à Scratch

Niveau : 1^{ère} Année Moyenne

Durée : 60 minutes

Enseignant : [Nom]

OBJECTIF GÉNÉRAL :

Réaliser un premier programme simple avec Scratch mettant en mouvement un sprite.

OBJECTIFS SPÉCIFIQUES :

1. Identifier l'interface Scratch (scène, sprite, blocs)
2. Assembler des blocs pour créer une séquence d'instructions
3. Exécuter le programme et observer le résultat

PRÉREQUIS :

- Savoir utiliser une souris
- Notions de base : avant/arrière, droite/gauche

MATÉRIEL :

- 1 ordinateur pour 2 élèves
- Accès à Scratch (en ligne ou installé)
- Fiche consigne imprimée
- Vidéo de démonstration (1 min)

DÉROULEMENT :

Phase 1 – Découverte guidée (15 min)

- Vidéo : "Qu'est-ce que Scratch ?" (5 min)
- En binômes : explorer l'interface librement (5 min)
- Mise en commun : nommer les zones principales (5 min)

Phase 2 – Défi "Le chat qui marche" (20 min)

- Consigne : "Faire avancer le chat de 100 pas puis dire Bonjour !"
- Aides différenciées :
 - * Niveau 1 : fiche avec les blocs à utiliser dessinés
 - * Niveau 2 : consigne orale uniquement
 - * Niveau 3 : ajouter un changement de couleur

Phase 3 – Partage et analyse (15 min)

- 3 binômes présentent leur solution au tableau
- Analyse collective : "Quels blocs avez-vous utilisés ?"

- Synth se : blocs Mouvement, Apparence,  v nements

Phase 4    D fi bonus (10 min)

- "Faire faire un carr  au chat"
- Les plus rapides aident les autres (tutorat)

 VALUATION :

- Formatrice : observation par l'enseignant (grille crit ri e)
- Les  l ves montrent leur programme au professeur
- Auto- valuation : "J'ai r ussi   ..." (3 items   cocher)

PROLONGEMENT :

- Proposition d'un d fi   la maison :
- "Cr er une histoire simple avec deux personnages qui dialoguent"

Exercice 3    Cr er une progression sur 3 s ances (15 min)

 nonc  :   partir de la fiche de s ance pr c dente, proposer une progression de 3 s ances Scratch pour des  l ves de 1AM.

S�ance	Titre	Objectif	Production
1	...	...	...
2	...	...	...
3	...	...	...

Corrig  :

S�ance	Titre	Objectif	Production
1	D�couverte	D�couvrir l'interface et cr�er un premier mouvement	Le chat qui marche
2	Interactions	Utiliser les blocs de contr�le (si, r�p�ter) et les capteurs	Questionnaire interactif
3	Projet	Concevoir un mini-jeu guid� (attraper la pomme)	Jeu "Attrape-pomme"

D tail S ance 2    "Questionnaire interactif" :

- Blocs : demander, si/alors, capteurs
- Exemple : poser une question    v rifier r ponse    dire "Bravo" ou "Essaie encore"
- Difficile : score avec variable

D tail S ance 3    "Attrape-pomme" :

- Sprite pomme qui tombe al atoirement
- Contr ler un panier avec les fl ches
- Score = nombre de pommes attrap es en 30 secondes
- Variable score + chronom tre

Exercice 4    Analyser une copie d' l ve (15 min)

 nonc  : Un  l ve a produit le programme Scratch suivant :

```

quand drapeau vert cliqu 
avancer de 10 pas
r p ter 10 fois
avancer de 5 pas
dire "Bonjour" pendant 1 secondes
attendre 2 secondes

```

```
aller x: 0 y: 0
```

1. Que fait ce programme ?
2. Repérer 3 erreurs de syntaxe ou logique
3. Proposer la version corrigée

Corrigé :

1. **Comportement** : Le chat avance de 10 pas, puis répète 10 fois (avancer 5 pas + dire Bonjour), puis attend 2 secondes et retourne au centre. **Mais** le répéter n'a pas de fin de répéter explicite, donc le programme est mal structuré.

2. Erreurs :

- **Structure** : Le bloc répéter n'englobe pas les instructions suivies (pas de fermeture). Scratch utilise des "chapeaux" visuels, donc en réalité c'est l'ordre d'empilement qui compte. La séquence réelle sera : avancer 10 → répéter 10 fois (avancer 5 + dire) → attendre → aller à (0,0).
- **Logique** : répéter 10 fois contient dire "Bonjour" pendant 1 secondes → prend 10 secondes. L'élève voulait peut-être le dire une seule fois.
- **Pédagogique** : la durée totale (10+ secondes) est trop longue pour un effet visible.

3. Version corrigée :

```
quand drapeau vert cliqué
  répéter 10 fois
    avancer de 10 pas
    attendre 0.5 secondes
    dire "Arrivé !" pendant 2 secondes
```

Exercice 5 – Concevoir une grille d'évaluation (15 min)

À noter : Concevoir une grille d'évaluation critique pour le projet "Attrape-pomme" (Séance 3). La grille doit comporter 4 critères avec 3 niveaux (Acquis, Partiellement acquis, Non acquis).

Corrigé :

Grille d'évaluation – Projet "Attrape-pomme"

Critère	Non acquis (0 pt)	Partiellement acquis (1 pt)	Acquis (2 pts)
Interface	Le projet ne contient qu'un sprite	Plusieurs sprites mais sans décoration	Sprites choisis
Mouvement	Le panier ne bouge pas	Le panier bouge mais ne suit pas les flèches	Le panier suit
Mécanique	La pomme ne tombe pas	La pomme tombe mais pas aléatoirement	La pomme tombe
Score	Pas de variable score	Variable score mais ne s'incrémente pas correctement	Score s'incrémente

Barème :

- 7-8 pts : Très bonne maîtrise
- 5-6 pts : Acquis
- 3-4 pts : Partiellement acquis
- 0-2 pts : À reprendre

Grille d'évaluation

Critère	Poids	Non acquis (0)	Partiel (0.5)	Acquis
Analyse d'exercice Scratch	20 %	Aucune analyse	Analyse partielle des blocs	Analyse
Conception de fiche de séance	20 %	Fiche incomplète ou incohérente	Fiche partiellement structurée	Fiche c
Progression sur 3 séances	20 %	Aucune progression	Progression sans lien entre séances	Progres
Analyse de copie d'élève	20 %	Erreurs non identifiées	Erreurs partielles	3 erreu
Conception de grille d'évaluation	20 %	Grille absente ou inutilisable	Grille avec critères incomplets	Grille c

Note pédagogique ENS

Transposition didactique : Scratch est un excellent outil pour introduire la programmation au collège (1AM) car il évite la syntaxe et permet une manipulation visuelle. Les concepts de séquence, boucle et condition sont abordés de manière intuitive.

Conseil pour l'enseignant formateur : Encourager l'approche "d'abord" pour motiver les élèves et privilégier le tutorat entre pairs. L'analyse d'erreurs dans les copies d'élèves est un excellent exercice de formation pour développer le regard pédagogique des futurs enseignants.

Prolongement : Demander aux stagiaires de concevoir une séquence complète de 6 séances Scratch avec évaluation diagnostique, formative et sommative.

Diagrammes conceptuels

Diagramme 1 à Concepts clés Scratch

```
mindmap
  root((Scratch))
    Mouvement
    Avancer
    Tourner
    Aller à
    Apparence
    Dire
    Penser
    Changer costume
    Contrôler
    Répéter
    Si / Alors
    Attendre
    Capteurs
    Demander
    Toucher
    Variables
    Score
    Chronomètre
    Événements
    Drapeau vert
    Touche pressée
```

Diagramme 2 à Déroulement d'une séance Scratch

```
graph TD
  A[Séance Scratch 1h] --> B[Phase 1 : Découverte guidée]
  A --> C[Phase 2 : D'abord principal]
  A --> D[Phase 3 : Partage et analyse]
  A --> E[Phase 4 : D'abord bonus]
  B --> F[Vidéo + exploration libre]
  C --> G[Le chat qui marche]
  C --> H[Niveau 1 : aide visuelle]
  C --> I[Niveau 2 : consigne orale]
```

```

C --> J[Niveau 3 : d'œfi supplœmentaire]
D --> K[Prœsentation des solutions]
D --> L[Synthœse collective]
E --> M[Tutorat entre pairs]
style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#e67e22,color:#fff
style D fill:#e74c3c,color:#fff
style E fill:#9b59b6,color:#fff

```

Dœroulement sœance œ 1h30 (90 min)

Phase	Durœe	Total cumulœ
Accueil	5 min	5 min
Exercice 1 œ Analyse Scratch	20 min	25 min
Exercice 2 œ Fiche de sœance	25 min	50 min
Exercice 3 œ Progression 3 sœances	15 min	65 min
Exercice 4 œ Analyse copie	15 min	80 min
Exercice 5 œ Grille d'œvaluation	10 min	90 min
Total	90 min	

Templates : Template/PG-fr.docx | Template/Presentation_BELACEL_V3.pptx

PDF : TD_Informatique_ENS_2œme_annœe_FR_09.pdf

TD Informatique â€” ENS 2^{ème} année â€” S^éance 09 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 2^{ème} année â€” D^épartement de fran^çais

Dur^ée : 1h30

Ann^ée universitaire : 2025-2026

Exercice 1 â€” D^éfinir le cahier des charges (20 min)

Ã€nonc^é : Vous devez d^évelopper une **application de gestion des notes** pour votre d^épartement. R^édig^éer les sections suivantes du cahier des charges :

1. **Contexte et objectifs** (2-3 phrases)
2. **Utilisateurs cibles** (qui utilise l'app ?)
3. **Fonctionnalit^{és} essentielles** (5 minimum)
4. **Contraintes techniques** (langage, base de donn^ées, plateforme)

Corrig^é :

CAHIER DES CHARGES â€” Gestion des Notes ENS

1. Contexte et objectifs

Le d^épartement de fran^çais de l'ENS Mostaganem souhaite informatiser le suivi des notes des Ã©tudiants pour les modules d'informatique. L'objectif est de remplacer les tableurs Excel par une application centrale.

2. Utilisateurs cibles

- Admin : gestion des utilisateurs, des mati^ères, des semestres
- Enseignant : saisie des notes, consultation des moyennes
- Ã©tudiant (lecture seule) : consultation de son relev^é

3. Fonctionnalit^{és} essentielles

- Authentification (login/mot de passe)
- CRUD Ã©tudiants, enseignants, mati^ères
- Saisie des notes avec coefficient
- Calcul automatique des moyennes (par mati^ère, semestre)
- G^én^ération de relev^é PDF
- Export CSV

4. Contraintes techniques

- Langage : Python (Django) ou Java (Spring Boot)
- Base : PostgreSQL ou MySQL
- Interface : Web responsive (Bootstrap)
- D^éploiement : serveur ENS ou cloud (H^ébergement institutionnel)

Exercice 2 â€” Conception UML : diagramme de classes (25 min)

Ã€nonc^é : Ã€ partir du cahier des charges pr^éc^édent, concevoir le diagramme de classes UML pour l'application de gestion des notes.

Classes Ã identifier :

- Utilisateur (classe abstraite)

- Admin, Enseignant, Etudiant (hÉRitage de Utilisateur)
- Matiere, Evaluation, Note
- Relations et multiplicité

Corrigé :

```

class Utilisateur {
    id: int;
    nom: string;
    prenom: string;
    email: string;
    mot_de_passe: string;
    se_connecter(): boolean;
    get_infos(): string;
}

class Admin extends Utilisateur {
    // ...
}

class Enseignant extends Utilisateur {
    // ...
}

class Etudiant extends Utilisateur {
    // ...
}

class Matiere {
    id: int;
    intitule: string;
    date: date;
    credit: int;
    type: string;
    coef: float;
    (DS, Examen, TP);
    get_moyenne(): float;
}

class Evaluation {
    id: int;
    date: date;
    type: string;
    coef: float;
    (DS, Examen, TP);
    get_moyenne(): float;
}

class Note {
    id: int;
    date: date;
    type: string;
    coef: float;
    (DS, Examen, TP);
    get_moyenne(): float;
}

```

Relations principales :

- Matiere 1 à * Evaluation (une matière a plusieurs Évaluations)
- Evaluation 1 à * Note (une Évaluation a plusieurs notes)

- Etudiant 1 à n * Note (un Étudiant a plusieurs notes)
- Enseignant 1 à n Matière (un enseignant peut avoir plusieurs matières)

Exercice 3 – Architecture et routes (15 min)

Nonc : Pour l'application web, définir :

1. L'architecture technique (3-tiers)
2. Les routes API REST (méthode + URL)
3. Les vues/templates minimaux

Corrigé :

1. Architecture 3-tiers :

```
[Client Navigateur]
↳ HTTP / HTTPS
[Backend Django / Spring]
↳ JDBC / ORM
[Base de données PostgreSQL]
```

2. Routes API REST :

Méthode	URL	Fonction
POST	/api/login	Authentification
GET	/api/etudiants	Liste Étudiants
GET	/api/etudiants/{id}	Détail Étudiant
POST	/api/etudiants	Créer Étudiant
GET	/api/matieres	Liste matières
GET	/api/matieres/{id}/moyenne	Moyenne matière
POST	/api/notes	Saisir note
GET	/api/etudiants/{id}/releve	Relevé PDF
GET	/api/export/csv	Export CSV

3. Vues minimales :

- Page de connexion (login.html)
- Dashboard enseignant (dashboard.html)
- Saisie de notes (saisie_notes.html)
- Relevé Étudiant (releve.html)
- Administration (admin.html)

Exercice 4 – Maquette wireframe (15 min)

Nonc : Dessiner (en texte) le wireframe de la page de **saisie de notes** pour un enseignant. La page doit contenir :

- Barre de navigation (haut)

- S  lecteur de mati  re + groupe
- Tableau : Noms | Pr  noms | Note
- Bouton "Enregistrer"
- Pied de page avec session active

CorrigÃ© :

[illegible]

â", [Logo] Gestion Notes ENS Bienvenue, Dr Belacel [DÃ©connexion] â",

[illegible] $\hat{a}'', \hat{a}'',$

â", Saisie des Notes â",

 $\hat{a}'', \hat{a}'',$

â", Matiere : [POO] Groupe : [ENS2-A] â",

 $\hat{a}'', \hat{a}'',$ [illegible]
$$\hat{a}'' , \hat{a}'' , N\hat{A}^\circ \hat{a}'' , \text{Nom } \hat{a}'' , \text{Pr}\hat{A}^\circ \text{nom } \hat{a}'' , \text{Note } \hat{a}'' , \hat{a}'' ,$$
[illegible]

â", â", 1 â", Benali â", Sara â", [15.00]â", â",

â", â", 2 â", Ziani â", Ahmed â", [12.50]â", â",

â", â", 3 â", Kaci â", Leila â", [17.00]â", â",

$$\hat{a}''_1, \hat{a}''_2, 4 \hat{a}''_3, \dots, \hat{a}''_n, \dots, \hat{a}''_{n+1}, [\dots] \hat{a}''_{n+2}, \hat{a}''_{n+3},$$
[illegible] $\hat{a}'', \hat{a}'',$

â", [Enregistrer] [Annuler] â",

 $\hat{a}'', \hat{a}'',$

â", DerniÃ"re sauvegarde : 15/03/2026 10:32 â",

[illegible]

â", Session : Enseignant â€" ENS Mostaganem â€" 2025/2026 â",

[illegible]

Exercice 5 – Plan de développement (jalons) (15 min)

Ã%noncÃ% : Ã%tablir un plan de dÃ%veloppement sur 6 semaines pour l'application. DÃ%finir 3 sprints de 2 semaines avec les livrables de chaque sprint.

CorrigÃ© :

PLAN DE DÉVELOPPEMENT " Sprint 0 À 3

SPRINT 0 (Semaine 1-2) " Fondations

J1-3 : Installation environnement (Django/Spring + PostgreSQL)

J4-7 : Modèle de données (classes entités)

J8-10 : Authentification + pages statiques

â€¦ Livrable : App avec login + navbar

SPRINT 1 (Semaine 3-4) – Fonctions principales

J11-13 : CRUD Ã©tudiants, enseignants, matiÃ¨res

J14-16 : Saisie des notes avec coefficients

J17-18 : Calcul automatique des moyennes

â€¦ Livrable : Saisie et consultation des notes

SPRINT 2 (Semaine 5-6) â€” Finalisation
 J19-20 : RelevÃ© de notes PDF
 J21-22 : Export CSV
 J23-24 : Tests utilisateurs + corrections
 J25-26 : DÃ©ploiement + documentation
 â€¦ Livrable : App livrÃ©e + formÃ©e

RÃ©PARTITION :
 - ModÃ¨le et back-end : 60 %
 - Front-end et UI : 25 %
 - Tests et dÃ©ploiement : 15 %

Grille d'Ã©valuation

CritÃ©re	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
Cahier des charges	20 %	Sections manquantes ou vagues	Partiellement rÃ©digÃ©	Complet, clair
Diagramme de classes UML	20 %	Absent ou erronÃ©	Classes identifiÃ©es sans relations	Classes + hÃ©rÃ©ditÃ©
Architecture et routes API	20 %	Architecture non dÃ©finie	Routes partielles	Architecture 3-tiers
Maquette wireframe	20 %	Absente ou illisible	Ã‰bauche de maquette	Wireframe dÃ©taillÃ©
Plan de dÃ©veloppement (sprints)	20 %	Aucun plan	Jalons partiels	3 sprints avec livrables

Note pÃ©dagogique ENS

Transposition didactique : La conception d'un cahier des charges et la modÃ©lisation UML peuvent Ãªtre transposÃ©es au lycÃ©e sous forme de projet fil rouge. Les Ã©lÃ©ves apprennent Ã structurer une pensÃ©e algorithmique et Ã anticiper les besoins utilisateurs avant de coder.

Conseil pour l'enseignant formateur : Insister sur l'importance de la phase de conception avant le dÃ©veloppement. Faire travailler les Ã©tudiants en binÃªmes sur des cas concrets (ex. application scolaire) pour rendre l'exercice authentique.

Prolongement : Proposer un mini-projet complet oÃ¹ les Ã©tudiants rÃ©alisent le diagramme de classes, implÃ©mentent le backend et prÃ©sentent leur travail lors d'une soutenance.

Diagrammes conceptuels

Diagramme 1 â€” Architecture 3-tiers

```
graph TB
  subgraph Client
    A[Navigateur Web]
  end
  subgraph Serveur
    B[Backend Django / Spring]
    C[Templates HTML]
    D[API REST]
  end
  subgraph Base
    E[PostgreSQL / MySQL]
  end
  A -->|HTTP/HTTPS| B
  B --> D
  B --> C
  B -->|JDBC/ORM| E
```

```

style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#e67e22,color:#fff
style D fill:#e74c3c,color:#fff
style E fill:#9b59b6,color:#fff

```

Diagramme 2 – Cycle de vie du projet

```

graph LR
A[Cahier des charges] --> B[Diagramme UML]
B --> C[Architecture API]
C --> D[Maquettage]
D --> E[Sprint 1 : Fondations]
E --> F[Sprint 2 : Fonctions principales]
F --> G[Sprint 3 : Finalisation]
G --> H[D ploiement]
style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#e67e22,color:#fff
style D fill:#e74c3c,color:#fff
style E fill:#e74c3c,color:#fff
style F fill:#e74c3c,color:#fff
style G fill:#9b59b6,color:#fff
style H fill:#2ecc71,color:#fff

```

D roulement s ance – 1h30 (90 min)

Phase	Dur�e	Total cumul�
Accueil	5 min	5 min
Exercice 1 – Cahier des charges	20 min	25 min
Exercice 2 – Diagramme de classes	25 min	50 min
Exercice 3 – Architecture et routes	15 min	65 min
Exercice 4 – Maquette wireframe	15 min	80 min
Exercice 5 – Plan de d�veloppement	10 min	90 min
Total	90 min	

Templates : Template/PG-fr.docx | Template/Presentation_BELACEL_V3.pptx

PDF : TD_Informatique_ENS_2 me_ann e_FR_10.pdf

TD Informatique â€” ENS 2^{ème} année â€” S^éance 10 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 2^{ème} année â€” D^épartement de fran^çais

Dur^ée : 1h30

Ann^ée universitaire : 2025-2026

Objectif de la s^éance

Finaliser le projet de gestion des notes entam^é à la s^éance pr^éc^édente :

- **Pr^ésensation par chaque groupe** (5 min)
- **D^émonstration fonctionnelle** (3 min)
- **Questions et ^évaluation par les pairs** (2 min)
- **Remise du code source et documentation**

Exercice 1 â€” Finalisation du code (20 min)

Chaque bin^{ôme} dispose de 20 minutes pour finaliser son application avant la pr^ésensation.

Checklist de Finalisation :

Tâche	Fait
CRUD ^é tudiants fonctionnel (cr ^é er, lire)	â€” ☐
Saisie de notes avec coefficients	â€” ☐
Calcul de moyenne (simple + pond ^é r ^é e)	â€” ☐
Au moins 3 ^é tudiants + 2 mati ^è res en base	â€” ☐
Interface responsive (CSS)	â€” ☐
Gestion des erreurs (note > 20, champs vides)	â€” ☐

Questions d'auto-^évaluation :

- Mon application g^ére-t-elle les cas limites (note n^égative, ^étudiant sans notes) ?
- Les donn^ées persistent-elles entre les sessions ?
- L'interface est-elle utilisable sur mobile ?

Exercice 2 â€” Pr^ésensation orale (25 min)

D^éroulement des pr^ésentations

Groupe	Cr ^é neau
Groupe 1 â€” Benali/Ziani	00:00 à 05:00
Groupe 2 â€” Kaci/Mokhtar	05:00 à 10:00

Groupe 3 â€” Amine/Fatima	10:00 â†’ 15:00
Groupe 4 â€” Youcef/Leila	15:00 â†’ 20:00
Groupe 5 â€” Hocine/Nadia	20:00 â†’ 25:00

Structure de la prÃ©sentation (5 min)

- PrÃ©sentation du projet (30 sec)
 - "Notre application permet de..."
- Architecture et choix techniques (1 min)
 - Langage, framework, base de donnÃ©es
 - "Nous avons choisi Django car..."
- DÃ©monstration (2 min)
 - Ajouter un Ã©tudiant
 - Saisir des notes
 - Afficher la moyenne
- DifficultÃ©s rencontrÃ©es (1 min)
 - "Le plus difficile a Ã©tÃ©..."
 - "Nous avons rÃ©solu en..."
- Conclusion et perspectives (30 sec)
 - "Si nous avions plus de temps, nous ajouterions..."

Grille d'Ã©valuation par les pairs

CritÃ©re	0	1	2
Application fonctionnelle	Pas de dÃ©mo	Partielle	ComplÃ©te
Code structurÃ©	DÃ©sordonnÃ©	Lisible	Propre + commentÃ©
PrÃ©sentation orale	Inaudible	Claire	Captivante
Gestion des erreurs	Aucune	Partielle	ComplÃ©te

Exercice 3 â€” Ã©valuation par les pairs (15 min)

Ã©noncÃ© : Pour chaque groupe (sauf le vÃ©tre), remplir la grille d'Ã©valuation ci-dessous.

Formulaire d'Ã©valuation :

GROUPE Ã©VALUÃ© : _____

CritÃ©res (1-4) :

- FonctionnalitÃ©s : ____ /4
- Interface utilisateur : ____ /4
- QualitÃ© du code : ____ /4
- PrÃ©sentation : ____ /4

Points forts :

- _____
- _____

Axes d'amÃ©lioration :

- _____
- _____

Note finale : ____ /20

Exercice 4 – Retrospective de projet (15 min)

À noter : En binôme, rédiger une retrospective (format "Keep, Problem, Try") de 10 lignes maximum.

Keep – Ce qui a bien fonctionné :

- _____
- _____
- _____

Problem – Les difficultés rencontrées :

- _____
- _____
- _____

Try – Ce que vous feriez différemment :

- _____
- _____
- _____

Exemple de retrospective :

KEEP :

- Le travail en binôme a permis de diviser le travail (front-end / back-end)
- Django facilite le CRUD avec les modèles

PROBLEM :

- La gestion des notes avec coefficients était complexe (calcul des moyennes)
- Nous avons sous-estimé le temps pour le déploiement

TRY :

- Faire un diagramme de classes avant de coder
- Prevoir plus de temps pour les tests utilisateurs

Exercice 5 – Remise et documentation (15 min)

À noter : Créer un fichier README.md pour le projet. Il doit contenir :

1. Titre du projet
2. Description (2-3 lignes)
3. Prérequis (Python, dépendances)
4. Installation et exécution (3-4 commandes)
5. Structure du projet (arbre des fichiers)
6. Auteurs

Corrigé :

```
# GestionNotesENS
```

```
Application web de gestion des notes pour le département
de français de l'ENS Mostaganem.
```

```

## PrÃ©requis

- Python 3.10+
- Django 4.2+
- PostgreSQL 14+

## Installation

git clone https://github.com/username/gestion-notes-ens.git
cd gestion-notes-ens
python -m venv venv
source venv/bin/activate # Windows : venv\Scripts\activate
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver

## Structure

gestion-notes-ens/
â"œâ"€â"€ manage.py
â"œâ"€â"€ requirements.txt
â"œâ"€â"€ README.md
â"œâ"€â"€ app/
â", â"œâ"€â"€ models.py # Classes : Utilisateur, Etudiant, Matiere, Note
â", â"œâ"€â"€ views.py # Routes et contrÃ´leurs
â", â"œâ"€â"€ urls.py # Configuration des URLs
â", â"œâ"€â"€ templates/ # HTML (base, login, dashboard, saisie, releve)
â", â"œâ"€â"€ static/ # CSS, JS
â"œâ"€â"€ notes/
â", â"œâ"€â"€ admin.py
â", â"œâ"€â"€ tests.py
â"œâ"€â"€ db.sqlite3

## Auteurs

- BENALI Sara â"œ" ENS Mostaganem
- ZIANI Ahmed â"œ" ENS Mostaganem

```

BarÃ©me de notation du projet

CritÃ©re	Poids	Points
FonctionnalitÃ©s (CRUD, saisie, moyenne)	40 %	/8

Qualité du code (structure, noms, lisibilité)	20 %	/4
Interface utilisateur (HTML/CSS, responsive)	15 %	/3
Présentation orale + démonstration	15 %	/3
Documentation (README, rétrospective)	10 %	/2
Total	100 %	20

Grille d'évaluation

Critère	Poids	Non acquis (0)	Partiel (0.5)
Finalisation du code (CRUD, notes, moyennes)	20 %	Application non fonctionnelle	Fonctionnalités partielles
Présentation orale et démonstration	20 %	Présentation absente ou incohérente	Présentation partielle
Évaluation par les pairs	20 %	Aucune évaluation rendue	Évaluations incomplètes
Rétrospective de projet (KPT)	20 %	Absente ou superficielle	Partiellement rédigée
Documentation et README	20 %	Absent ou incomplet	README partiel

Note pédagogique ENS

Transposition didactique : La soutenance de projet est transposable au lycée sous forme d'exposé oral où les élèves présentent leur site web ou leur jeu Scratch. Cela développe des compétences transversales (communication, esprit critique, auto-évaluation).

Conseil pour l'enseignant formateur : Structurer les présentations avec un temps limité pour habituer les étudiants aux contraintes professionnelles. La grille d'évaluation par les pairs responsabilise et développe le regard critique.

Prolongement : Organiser un "mini-concours" où un jury extérieur (autres enseignants, professionnels) évalue les projets et remet des prix.

Diagrammes conceptuels

Diagramme 1 : Déroulement des présentations

```
graph TD
    A[Soutenance 5 min] --> B[1. Présentation du projet]
    A --> C[2. Architecture et choix techniques]
    A --> D[3. Démonstration]
    A --> E[4. Difficultés rencontrées]
    A --> F[5. Conclusion et perspectives]
    B --> G["30 sec"]
    C --> H["1 min"]
    D --> I["2 min"]
    E --> J["1 min"]
    F --> K["30 sec"]
    style A fill:#4a90d9,color:#fff
    style B fill:#2ecc71,color:#fff
    style C fill:#e67e22,color:#fff
    style D fill:#e74c3c,color:#fff
    style E fill:#9b59b6,color:#fff
    style F fill:#4a90d9,color:#fff
```

Diagramme 2 : Évaluation de projet

```
mindmap
    root((Évaluation Projet))
```

Fonctionnalités
 CRUD
 Saisie notes
 Calcul moyennes
 Gestion erreurs
 Qualité du code
 Structure
 Nommage
 Lisibilité
 Interface utilisateur
 HTML/CSS
 Responsive
 Accessibilité
 Présentation
 Clarté
 Démonstration
 Gestion du temps
 Documentation
 README
 Retrospective
 Code commenté

Déroulement séance de 1h30 (90 min)

Phase	Durée	Total cumulé
Accueil + consignes	5 min	5 min
Exercice 1 de Finalisation	20 min	25 min
Exercice 2 de Présentations (5 groupes de 5 min)	25 min	50 min
Exercice 3 de Évaluation par les pairs	15 min	65 min
Exercice 4 de Retrospective	15 min	80 min
Exercice 5 de README + remise	10 min	90 min
Total	90 min	